

DATAEXPLORE '25

Team - DXP01

# Enhancing Hotel Revenue Management Through Predictive Analytics

# Table of Contents

|                                                            |    |
|------------------------------------------------------------|----|
| Abstract.....                                              | 3  |
| Introduction.....                                          | 4  |
| Methods .....                                              | 4  |
| Inspecting data structure .....                            | 4  |
| Summary statistics .....                                   | 6  |
| Numerical.....                                             | 6  |
| Categorical .....                                          | 7  |
| Analyze target variable .....                              | 8  |
| Replacing inconsistent naming in the training dataset..... | 9  |
| Handle duplicates and check unique values.....             | 9  |
| Training dataset.....                                      | 9  |
| Testing dataset (validation dataset) .....                 | 11 |
| Handling null values .....                                 | 12 |
| .....                                                      | 12 |
| Change data types .....                                    | 12 |
| Feature extraction.....                                    | 12 |
| Feature Engineering .....                                  | 12 |
| Numerical feature analysis.....                            | 14 |
| Univariate analysis .....                                  | 14 |
| Multivariate analysis .....                                | 17 |
| Reservation Status, grouped by a categorical feature ..... | 18 |
| Reservation status by hotel type and promotion use .....   | 18 |
| Categorical Feature Analysis .....                         | 19 |
| Outlier detection and data quality check.....              | 23 |
| Feature Distributions by Reservation Type.....             | 25 |
| Correlation Analysis.....                                  | 28 |
| Time Series Analysis.....                                  | 32 |
| Analysis by Hotel Type.....                                | 36 |
| Feature Selection.....                                     | 44 |
| Model training and Evaluation.....                         | 47 |
| Random Forest.....                                         | 47 |
| 1. Random Forest Without Balancing Class Weights.....      | 47 |

|                                                   |                                     |
|---------------------------------------------------|-------------------------------------|
| 2. Random Forest with Balanced Class Weights..... | 47                                  |
| 3. Random Forest with feature selection.....      | 48                                  |
| 4. Random Forest with Oversampling .....          | 48                                  |
| XGBoost .....                                     | 48                                  |
| 1. Base XGBoost Model .....                       | 49                                  |
| 2. XGBoost with Balanced Class Weights .....      | 49                                  |
| LightBGM.....                                     | 49                                  |
| Logistic Regression.....                          | 50                                  |
| MLP Classifier .....                              | 51                                  |
| Hyperparameter Tuning .....                       | 53                                  |
| Model Deployment .....                            | 55                                  |
| Conclusion .....                                  | 58                                  |
| Acknowledgments.....                              | <b>Error! Bookmark not defined.</b> |
| References.....                                   | <b>Error! Bookmark not defined.</b> |

## Abstract

The hotel industry faces significant challenges from booking cancellations and no-shows, leading to revenue loss and operational inefficiencies. This study investigates these issues for Hotel A, a

chain operating airport hotels, resorts, and city hotels, using a data-driven approach to enhance revenue management. Leveraging a dataset of 27,500 booking records, we conducted an exploratory data analysis (EDA) to uncover patterns in reservation statuses (Check-In, Cancel, No-Show), identifying key factors such as lead time, hotel type, and booking channel as potential drivers of cancellations and no-shows. Revenue losses were quantified by analyzing room rates and stay durations associated with unfulfilled bookings. Subsequently, several predictive models were developed to classify reservation outcomes, with performance evaluated using the F1-score to address class imbalance. Customer segmentation was explored to characterize guest behavior, enhancing interpretability through feature importance and probability outputs. This analysis provides actionable insights for Hotel A's management to optimize resource allocation, refine cancellation policies, and mitigate financial impacts, demonstrating the power of predictive analytics in hospitality management.

## Introduction

The hotel industry is a growing industry that caters to many customers, and travelers in need of long-term or short-term accommodation. But due to several reasons, customers tend to make booking cancellations or no-shows. In our case study, we focus on the 3 main hotel types.

1. Airport hotels near airports offer full service to shorter-term travelers.
2. Resort- Luxury accommodation with long-term customers.
3. City hotels- Situated in business areas and accommodate corporate travelers.

However, the unpredicted results cause hotels to face significant challenges like revenue losses, insufficient resource utilization, and other challenges, etc. Therefore, our team comes with advanced data analytical methods like Exploratory data analytics, hypothesis testing, identifying important features, predictive modeling, and customer segmentation to mitigate the issues and identify the hidden patterns to come up with the best solutions for hotel management. The dataset contains 27500 data points and 2749 for validating data points. The training data set contains over 20 attributes. For each hotel, they might have to face unique types of challenges according to their hotel type.

Our goal is to address this problem and come up with a solution by using statistical approaches and machine learning techniques and observing historical booking data, reservation details, customer demographics, payment methods, etc.

## Methods

### Inspecting data structure

At the beginning of the project, we loaded the data set from the provided sources by using the Pandas data frame. The training dataset was renamed as `df_train`, and the test dataset was renamed as `df_test`. The following is an overview of both datasets.

First off we identified that babies column was mostly null through initial inspection. Then replaced all train nulls in babies column with 0

```
df_train["Babies"] = df_train["Babies"].fillna(0)
df_test["Babies"] = df_test["Babies"].fillna(0)
```

For the training dataset after replacing null babies column values,

| Column                 | Non-Null Count | Dtype   |
|------------------------|----------------|---------|
| Reservation-id         | 27499          | int64   |
| Gender                 | 27499          | object  |
| Age                    | 27499          | int64   |
| Ethnicity              | 27499          | object  |
| Educational_Level      | 27499          | object  |
| Income                 | 27499          | object  |
| Country_region         | 27499          | object  |
| Hotel_Type             | 27499          | object  |
| Expected_checkin       | 27499          | object  |
| Expected_checkout      | 27499          | object  |
| Booking_date           | 27499          | object  |
| Adults                 | 27499          | int64   |
| Children               | 27499          | int64   |
| Babies                 | 22748          | float64 |
| Meal_Type              | 27499          | object  |
| Visited_Previously     | 27499          | object  |
| Previous_Cancellations | 27499          | object  |
| Deposit_type           | 27499          | object  |

|                      |       |         |
|----------------------|-------|---------|
| Booking_channel      | 27499 | object  |
| Required_Car_Parking | 27499 | object  |
| Reservation_Status   | 27499 | object  |
| Use_Promotion        | 27499 | object  |
| Discount_Rate        | 27499 | float64 |
| Room_Rate            | 27499 | float64 |

The target variable is reservation status which contains 3 main types. We are to give a solution to accurately assess which users are most likely to be checking in, no-show, and cancel their bookings.

The summary statistics are taken to identify the statistical characteristics in both numerical and categorical data.

## Summary statistics

### Numerical

Age, adults, children, babies, discount rate, and room rate are put under the numerical statistical summary. For the training dataset, Age is distributed between 18 to 70 years old. The mean age is 44 years old. The number of adults in a group lies between 1 to 5 member groups and the mean number is about 2 members.

The maximum number of children in a group is 3 and the minimum number is 1, while the average 1.7 number of children can appear in a group. The maximum number for babies is 2 and the minimum can be 0. The average is 0. The discount rates are distributed from 0 to 30. Half of the reservations have a discount of 5% or less, while 75% have a discount of 10% or less. The average room rate is 155.65, with prices ranging from 100 to 250.

| Statistic | Reservation-id | Age     | Adults  | Children | Babies  | Discount_Rate | Room_Rate |
|-----------|----------------|---------|---------|----------|---------|---------------|-----------|
| count     | 27499.0        | 27499.0 | 27499.0 | 27499.0  | 22744.0 | 27499.0       | 27499.0   |
| mean      | 5.016461e+07   | 43.9775 | 2.2321  | 1.7439   | 0.3742  | 8.2930        | 155.6451  |
| std       | 2.886911e+07   | 15.3031 | 1.1802  | 0.7220   | 0.6184  | 7.8417        | 38.3148   |

|     |              |         |        |        |        |         |          |
|-----|--------------|---------|--------|--------|--------|---------|----------|
| min | 3.154000e+03 | 18.0000 | 1.0000 | 1.0000 | 0.0000 | 0.0000  | 100.0000 |
| 25% | 2.523943e+07 | 31.0000 | 1.0000 | 1.0000 | 0.0000 | 0.0000  | 124.0000 |
| 50% | 5.014996e+07 | 44.0000 | 2.0000 | 2.0000 | 0.0000 | 5.0000  | 149.4530 |
| 75% | 7.518288e+07 | 57.0000 | 3.0000 | 2.0000 | 1.0000 | 10.0000 | 181.0000 |
| max | 9.999900e+07 | 70.0000 | 5.0000 | 3.0000 | 2.0000 | 30.0000 | 250.0000 |

When we observed the training and testing datasets, we identified that the training dataset includes roughly over 90% of the total data, therefore we can come to conclusions about the entire population based on it.

Thus almost all EDA steps have been performed on the training data (df\_train) and validation dataset was only used when we wanted to apply preprocessing steps uniformly.

## Categorical

There are a total 17 number of categorical variables. And the summary statistics of categorical features are as following

| Attribute          | Count | Unique | Top Value   | Frequency |
|--------------------|-------|--------|-------------|-----------|
| Gender             | 27499 | 2      | F           | 14902     |
| Ethnicity          | 27499 | 4      | Latino      | 7909      |
| Educational Level  | 27499 | 4      | College     | 11924     |
| Income             | 27499 | 4      | 50K -- 100K | 7936      |
| Country Region     | 27499 | 4      | South       | 10917     |
| Hotel Type         | 27499 | 3      | City Hotel  | 11731     |
| Expected Check-in  | 27499 | 901    | 2016-02-09  | 145       |
| Expected Check-out | 27499 | 929    | 2016-02-10  | 91        |
| Booking Date       | 27499 | 1133   | 2015-10-19  | 232       |

|                        |       |   |            |       |
|------------------------|-------|---|------------|-------|
| Meal Type              | 27499 | 3 | BB         | 14224 |
| Visited Previously     | 27499 | 2 | No         | 18344 |
| Previous Cancellations | 27499 | 2 | No         | 24379 |
| Deposit Type           | 27499 | 3 | No Deposit | 14791 |
| Booking Channel        | 27499 | 3 | Online     | 15574 |
| Required Car Parking   | 27499 | 2 | Yes        | 19629 |
| Reservation Status     | 27499 | 3 | Check-out  | 21240 |
| Use Promotion          | 27499 | 2 | Yes        | 16230 |

## Analyze target variable

The check-out canceled and no-show counts in the training dataset are shown in the following table.

| Reservation_Status | Count |
|--------------------|-------|
| check-out          | 21240 |
| canceled           | 4134  |
| no-show            | 2125  |

The check-out canceled and no-show counts in the testing dataset are shown in the following table.

| Reservation_Status | Count |
|--------------------|-------|
| check-in           | 1610  |
| canceled           | 741   |
| no-show            | 398   |

Therefore, we can come to understand that this is a dataset with moderate imbalance. Check-in is the majority class and the no-show class has a significantly smaller number of data points.



## Replacing inconsistent naming in the training dataset

In further analysis we identified the unique data values in the target in Reservation Status has been set inconsistently between the train and test dataset.

The values check-in and check-out have been used to denote the same thing in the two datasets. Therefore, all the check-outs did not align with our final goal so check-out was transformed to Check-in.

```
df_train['Reservation_Status'] = df_train['Reservation_Status'].replace('check-out', 'check-in')
```

| Reservation_Status | Count |
|--------------------|-------|
| check-in           | 1610  |
| canceled           | 741   |
| no-show            | 398   |

## Handle duplicates and check unique values.

### Training dataset

Firstly, we handle duplicates and preprocessing in the training dataset. However, we didn't identify any duplicates in that dataset.

### Dropping Reservation\_ID column

Then we dropped the reservation ID column in both the training and testing datasets. Because it is not useful for our analysis and model building. The unique values are used to identify how many data types are contained in each column. Therefore, we could get a rough idea of how many data types are dealing with.

### Inspecting unique values

The unique values of high cardinality are together in an array format below which do not include all values.

| Attribute | Unique Values                                                                                                        |
|-----------|----------------------------------------------------------------------------------------------------------------------|
| Gender    | [M, F]                                                                                                               |
| Age       | [40, 49, 42, 25, 62, 34, 53, 59, 21, 69, 52, 51, 22, 18, 28, 41, 33, 44, 60, 45, 23, 24, 38, 26, 37, 30, 27, 56, 35] |

|                        |                                                                                                                          |
|------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Ethnicity              | [Latino, African American, Caucasian, Asian American, Other]                                                             |
| Educational_Level      | [High-School, College, Grad, Mid-School]                                                                                 |
| Income                 | [<25K, 50K--100K, >100K, 25K--50K]                                                                                       |
| Country_region         | [North, East, South, West]                                                                                               |
| Hotel_Type             | [City Hotel, Airport Hotels, Resort]                                                                                     |
| Expected_checkin       | [2015-07-01, 2015-07-02, 2015-07-03, 2015-05-01, 2015-06-10, 2015-06-05, 2015-06-12, 2015-06-15, 2015-07-06, 2015-07-07] |
| Expected_checkout      | [2015-07-02, 2015-07-06, 2015-07-03, 2015-07-07, 2015-07-10, 2015-07-08, 2015-07-05]                                     |
| Booking_date           | [2015-05-21, 2015-05-26, 2015-06-29, 2015-06-21, 2015-06-30, 2015-06-05, 2015-05-31, 2015-06-19, 2015-06-12]             |
| Adults                 | [4, 1, 3, 2, 5]                                                                                                          |
| Children               | [2, 3, 1]                                                                                                                |
| Babies                 | [nan, 0.0, 2.0, 1.0]                                                                                                     |
| Meal_Type              | [BB, FB, HB]                                                                                                             |
| Visited_Previously     | [No, Yes]                                                                                                                |
| Previous_Cancellations | [No, Yes]                                                                                                                |
| Deposit_type           | [No Deposit, Non-Refundable, Refundable]                                                                                 |
| Booking_channel        | [Online, Agent, Direct]                                                                                                  |
| Required_Car_Parking   | [Yes, No]                                                                                                                |
| Reservation_Status     | [check-in, canceled, no-show]                                                                                            |
| Use_Promotion          | [Yes, No]                                                                                                                |
| Discount_Rate          | [0, 5, 10, 25, 20, 15, 30]                                                                                               |

|           |                                                                                                                                                          |
|-----------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Room_Rate | [106.44, 158.04, 212.61, 185.95, 109.12, 178.45, 130.55, 191.32, 162.75, 199.87, 143.98, 158.56, 124.03, 187.55, 138.79, 145.50, 166.99, 139.50, 190.00] |
|-----------|----------------------------------------------------------------------------------------------------------------------------------------------------------|

### Testing dataset (validation dataset)

According to the observations, there are no differences between training and testing data when it comes to the count of unique values in categorical columns. Meaning that there are no data categories in testing dataset that's not in the training data.

| Attribute              | Count |
|------------------------|-------|
| Gender                 | 2     |
| Age                    | 53    |
| Ethnicity              | 4     |
| Educational_Level      | 4     |
| Income                 | 4     |
| Country_region         | 4     |
| Hotel_Type             | 3     |
| Expected_checkin       | 151   |
| Expected_checkout      | 156   |
| Booking_date           | 322   |
| Adults                 | 5     |
| Children               | 3     |
| Babies                 | 3     |
| Meal_Type              | 3     |
| Visited_Previously     | 2     |
| Previous_Cancellations | 2     |
| Deposit_type           | 3     |

|                      |      |
|----------------------|------|
| Booking_channel      | 3    |
| Required_Car_Parking | 2    |
| Reservation_Status   | 3    |
| Use_Promotion        | 2    |
| Discount_Rate        | 7    |
| Room_Rate            | 2269 |

## Handling null values

We've already identified and removed and replaced the null in the babies column in a previous step and they were applied to the both test and train dataset. Thus we can be confident of our dataset not having null values going forwards.

## Change data types

Data types were changed as appropriate. Some of the given data features were not in an ideal format, so we changed Babies to integer, discount rate to float, booking date, expected checking, and expected checkout to DateTime. The changes were applied to both datasets.

## Feature extraction

The following new features were created for both the training (df\_train) and test (df\_test) datasets

1. Expected\_Checkin\_Year: This feature extracts the year of the expected check-in date. It provides a high-level overview of temporal trends over different years and could be useful in capturing annual variations in hotel booking patterns.
2. Expected\_Checkin\_Month: This feature captures the month of the year (from 1 to 12). By isolating the month, the model can identify patterns that may exist in certain months, such as increased bookings during holidays or peak seasons.
3. Expected\_Checkin\_Day\_of\_Week: This feature extracts the day of the week (from 0 to 6, where Monday is 0 and Sunday is 6). This temporal feature helps identify trends related to weekday versus weekend bookings, which can be critical for understanding customer behavior and hotel demand.

## Feature Engineering

The following features were engineered to improve o

**Length of Stay:** The length of stay for each reservation was calculated by subtracting the 'Expected\_checkin' date from the 'Expected\_checkout' date. This feature represents the duration of the stay and is important for understanding customer behavior and demand for rooms.

```
# Calculate the length of stay
df_train['Length_of_Stay'] = (df_train['Expected_checkout'] - df_train['Expected_checkin']).dt.days
df_test['Length_of_Stay'] = (df_test['Expected_checkout'] - df_test['Expected_checkin']).dt.days
```

### Lead Time (Lead Days)

Lead Time measures the number of days between the booking date and the expected check-in date. This feature is valuable for understanding booking patterns and predicting the lead time for different customer segments.

```
#calculate the lead time - how many days were spent between the date of booking and the date of expected check-in
df_train['Lead_Days'] = (df_train['Expected_checkin'] - df_train['Booking_date']).dt.days
df_test['Lead_Days'] = (df_test['Expected_checkin'] - df_test['Booking_date']).dt.days
```

### Age Group

To handle the 'Age' feature and prevent high cardinality, Age Groups were created by binning the age values into specific ranges. This categorization helps the model better understand customer demographics by grouping individuals into similar age categories.

The bins were defined as

```
18-25', '26-35', '36-45', '46-55', '56-65', '66-70'
```

### Travel Group Size

The Travel Group Size feature was derived by adding the number of Adults and Children in a reservation. This provides insight into the size of the party, which influences the booking's total cost and room requirements

```
# Travel Group Size feature by adding the number of Adults and Children
df_train['Group_Size'] = (df_train['Adults'] + df_train['Children'])
df_test['Group_Size'] = (df_test['Adults'] + df_test['Children'])
```

### Total Cost

The Total Cost of the reservation was calculated by multiplying the Length of Stay by the Room Rate. we multiplied the room cost by another room whenever the group size exceeded 5.

```
# Calculate the total cost
df_train['Total_Cost'] = (df_train['Length_of_Stay'] * df_train['Room_Rate'])
df_train.loc[df_train['Group_Size'] > 5, 'Total_Cost'] = df_train['Total_Cost'] * 2 # Multiply by 2
```

```
df_train.loc[df_train['Group_Size'] > 10, 'Total_Cost'] = df_train['Total_Cost'] * 3
```

### Final Cost (after Discount)

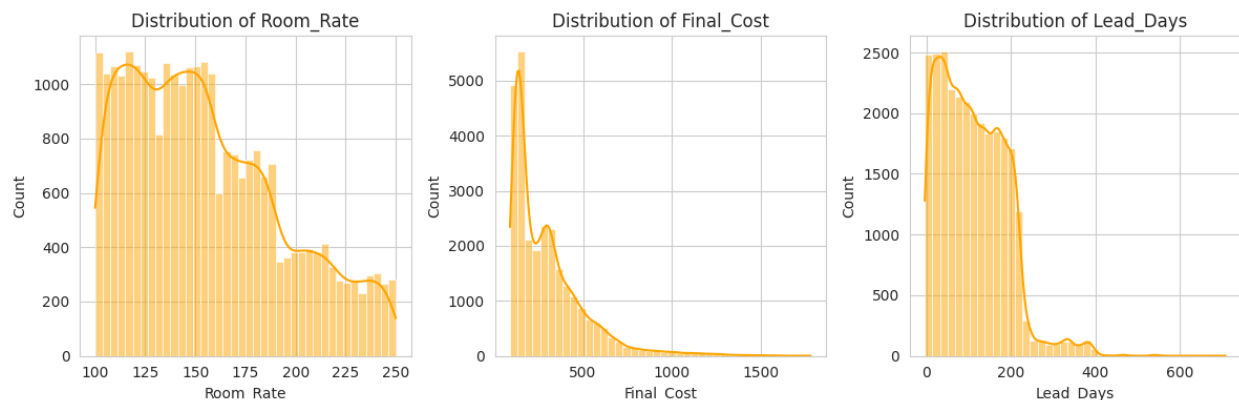
The Final Cost was calculated by applying the discount rate to the Total Cost. This adjusted cost provides the actual amount to be paid after considering any available discounts

```
# Calculate the final cost after discount
df_train['Final_Cost'] = df_train['Total_Cost'] - (df_train['Total_Cost'] * df_train['Discount_Rate']/100)
```

## Numerical feature analysis

### Univariate analysis

Room rate, final cost, and lead days

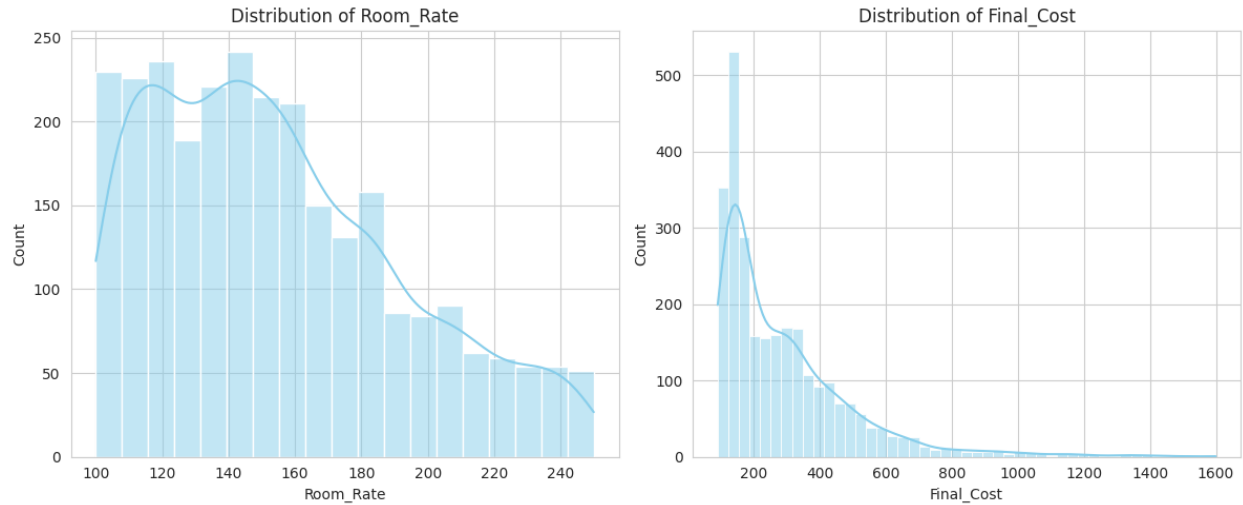


Room rate, final cost, and lead days were selected for univariate analysis. The distribution of the room rates is slightly skewed, which means the room rates are concentrated in lower room rates rather than higher room rates because higher room rates attract a smaller number of customers. Some peaks in room rates show that some room price ranges are more popular than others.

The distribution of the final cost is heavily right skewed as there's always several high profile travelers that spend a lot while most spend a moderate amount.

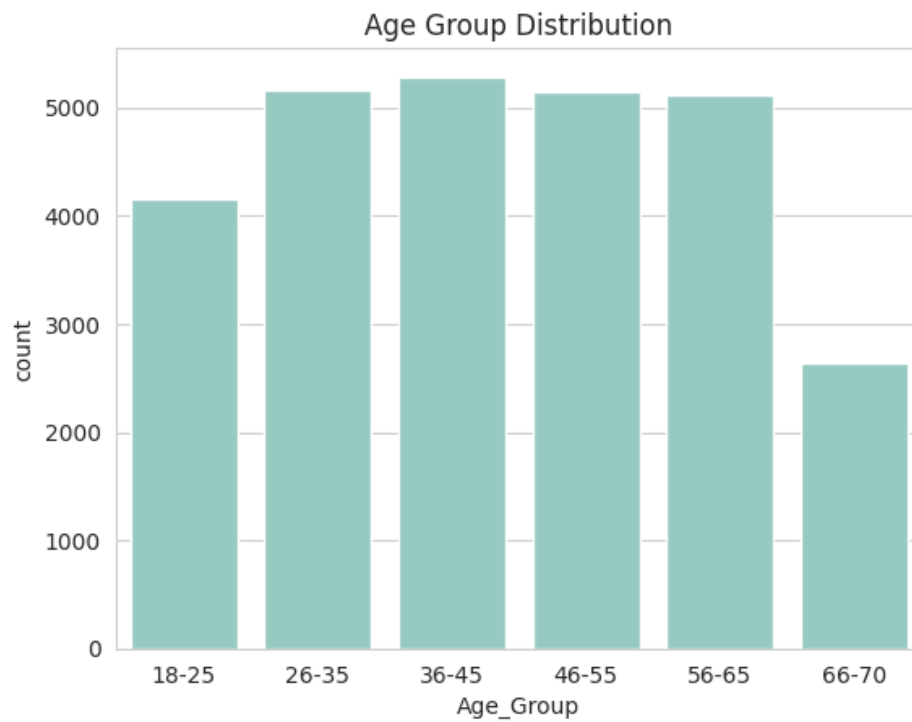
Most bookings are recorded around lower prices, meaning the guests are more likely to purchase lower prices or apply discounts. High final cost only recorded for less frequent of time. The idea of this chart indicates that more guests tend to choose cost-efficient purchases.

The lead day's chart has a bimodal distribution where there are 2 noticeable peaks. The 0-50 range indicates that many customers made bookings. After that, the trend is going downward. A smaller peak in 150-200 suggests some customers get their bookings in that range. After 200 the trend is the booking count get lower numbers. The count of over 400 is very rare.



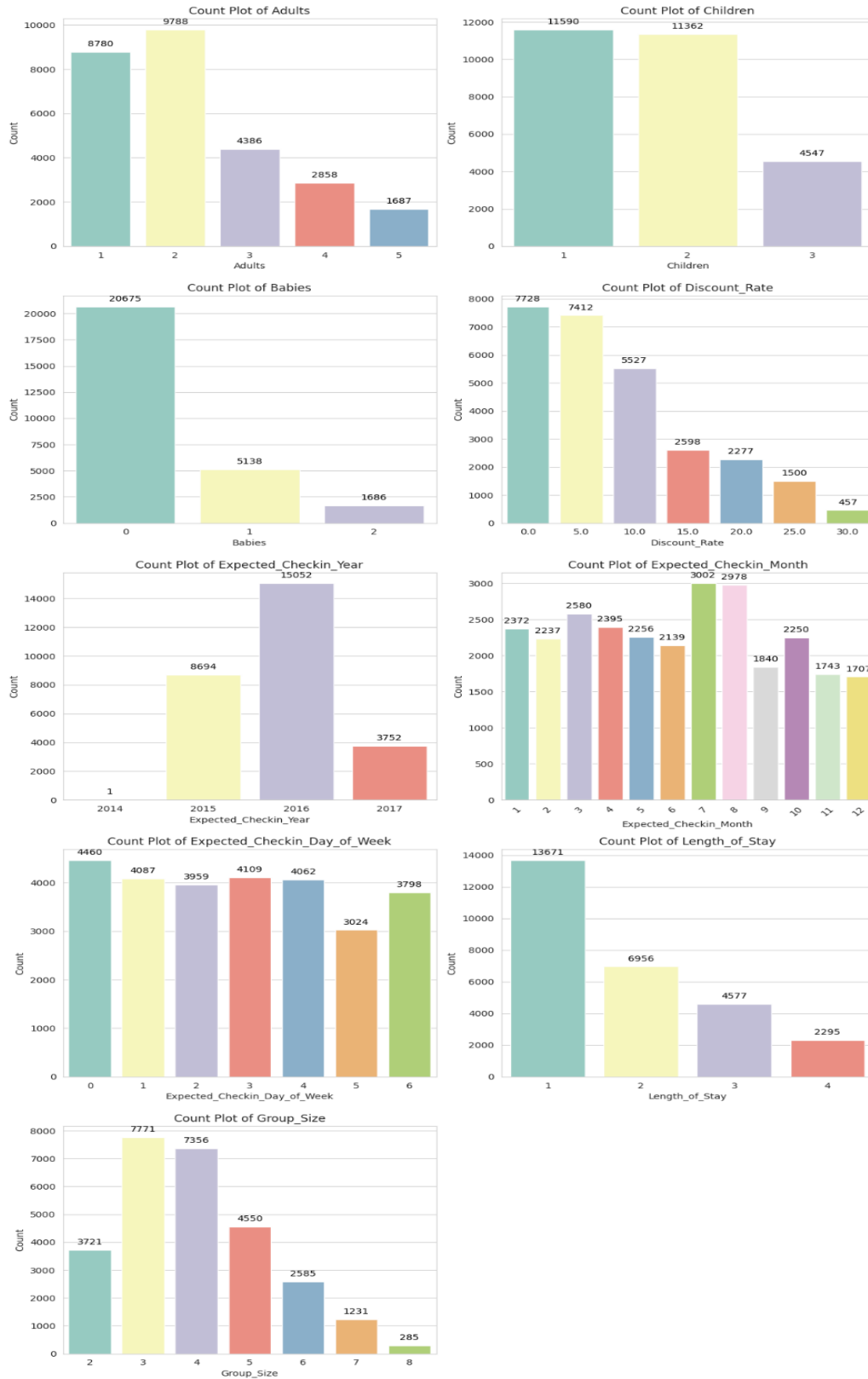
The distribution is similar for the test dataset as in the training data. Thus, we will be doing further plotting using the training dataset.

Age group distribution



Categorize the age groups to 18-25,26-35,36-45,56-66, and 66-70. With this approach, we can get some useful insides for our model building and feature importance.

Univariate analysis for other variables



Count plot for adults most bookings involve 1 or 2 adults. The 2 adults are the most common in the booking. The number is shown in 9788 bookings. 3,4,5 number of the adults in the group is significantly



lower compared to others. This indicates small groups are the primary customers. Businesses can more focus on small groups or couples for more profits.

Children count distribution majority have 1 or 2 children and few number of the having 3 children. Most of the time the guests do not have babies with them. That number is 20675. 5138 bookings include 1 baby, and 1686 counts have 2 babies.

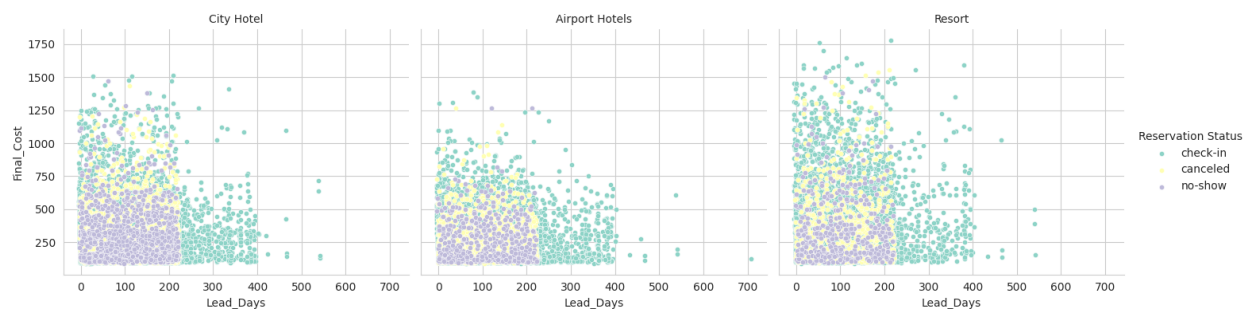
Discount rate distribution, there are the highest numbers of 0% discount bookings, and it is 7728. 5%-10% discount rates are more common in the bookings. More than 10% is much rarer among the bookings. The expected check-in-year distribution, in 2016 peaked in check-ins, and after that check-in declined in 2017. If 2017 shows some decline, the business may need to adopt new marketing strategies. Expected check-in month distribution, in July and August have a high number of bookings while other month bookings are evenly distributed in the chart. Therefore, we can identify seasonality among the bookings and a seasonal marketing campaign can earn more money.

On the expected check-in day of the week, Monday and Tuesday are the highest in the week, while Saturday is the lowest in the week. The length of the stay, the maximum duration is 1 night, and 2 nights is the second maximum length of stay. 3-4 days of stay is not much common. Most groups have 3-4 individuals while larger groups with 6-8 people is very rare. The least number of guests spent more than 8 days.

Small groups and short stays are more common in the bookings. Families with babies are rare customers and Most bookings happen without heavy discounts. The hotel could boost weekend and longer-stay bookings through targeted promotions to gain more profits.

## Multivariate analysis

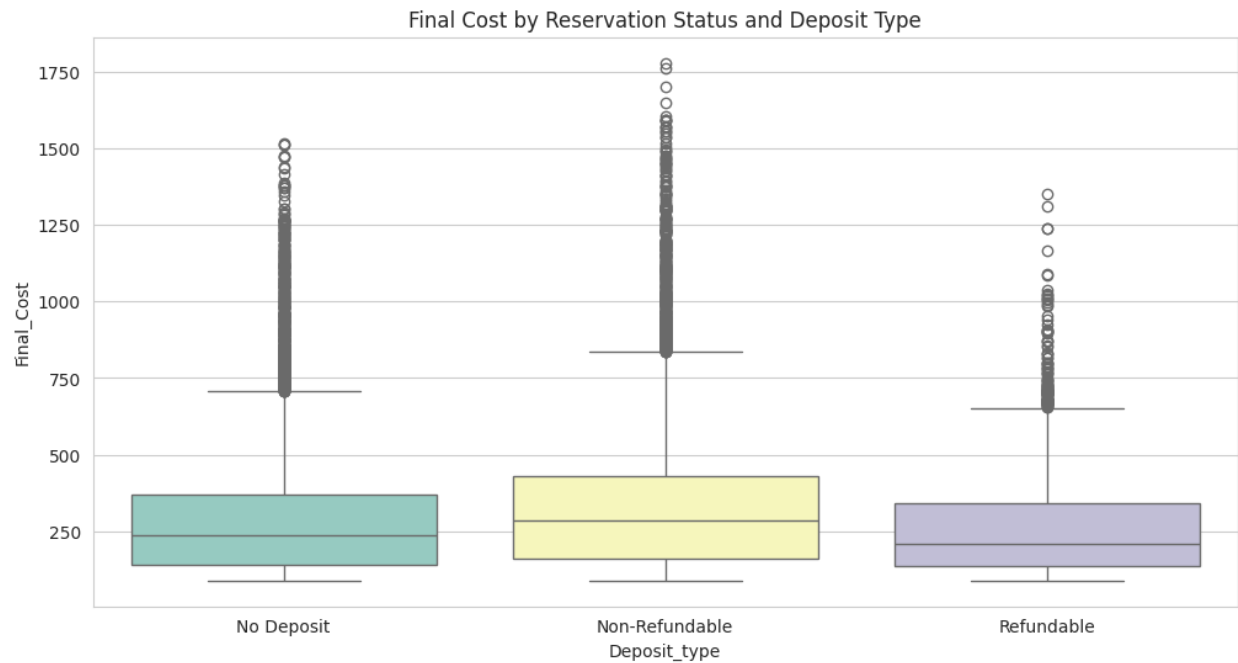
### Reservation Status and split by Hotel Type.



Most reservations happen within 0–200 lead days, with fewer bookings beyond that range. High-cost bookings lead to lower lead days. This means customers pay more to book closer to check-in. Cancellations (yellow) and No-shows (purple) are concentrated in lower-cost bookings.

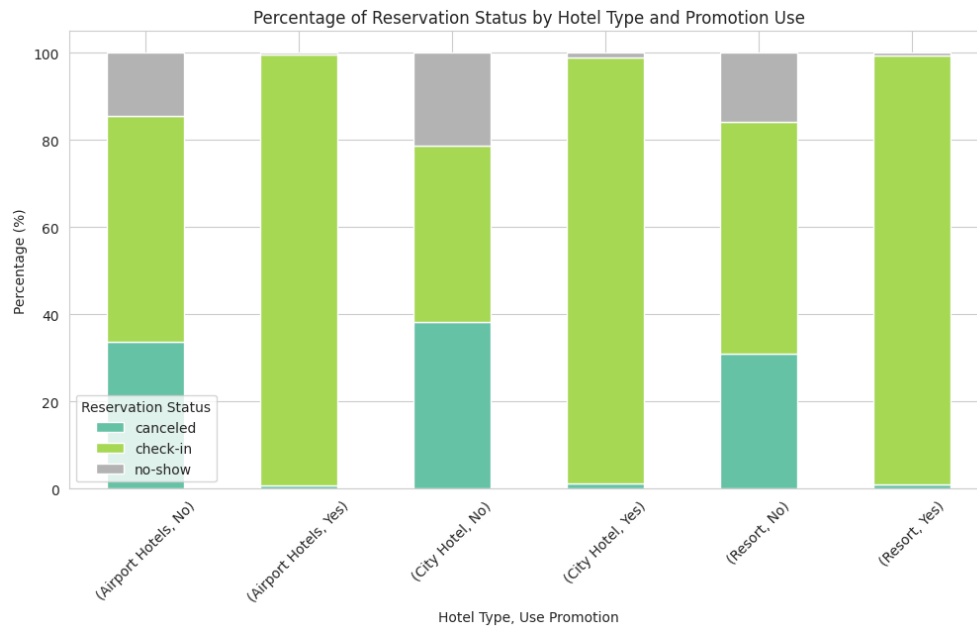
City hotel customers concentrate on the high number of bookings and the lower lead days the high number of cancellations are lower than 750. Only a few high-cost bookings. airport hotels is similar to city hotels but with fewer long lead time booking. High-cost bookings check-in successfully and cancellations and no shows are still prominent in lower-cost bookings. Resort hotels there are more long-time bookings compared to other hotel types. Higher final costs bookings have a broader range of lead days. Cancellations are frequent within the 200-400 days.

## Reservation Status, grouped by a categorical feature



All the deposit types are distributed around 250-300. Which indicates that there is a little difference between average deposit types. According to the interquartile range, 50% of data lies between 200-400 across all categories. There are number of outliers detected by the box plot, but when it comes to the real world we identified that those final costs are important for predictions.

## Reservation status by hotel type and promotion use



According to the chart, in most of the bookings across all hotel types, most reservations result in check-ins and cancellations and no-show results are lower than check-ins.

Airport hotels, city hotels, and resorts with promotions tend to dominate with check-ins with minimum cancellations and no-shows. Without promotions, airport hotel check-ins remain high, and no-cancellation and cancellations are slightly increased. City hotels without promotions, check-ins and cancellations have similar distribution and no-shows are less than that. Resorts have several check-ins and cancellations and no-shows are lower than check-ins.

## Categorical Feature Analysis

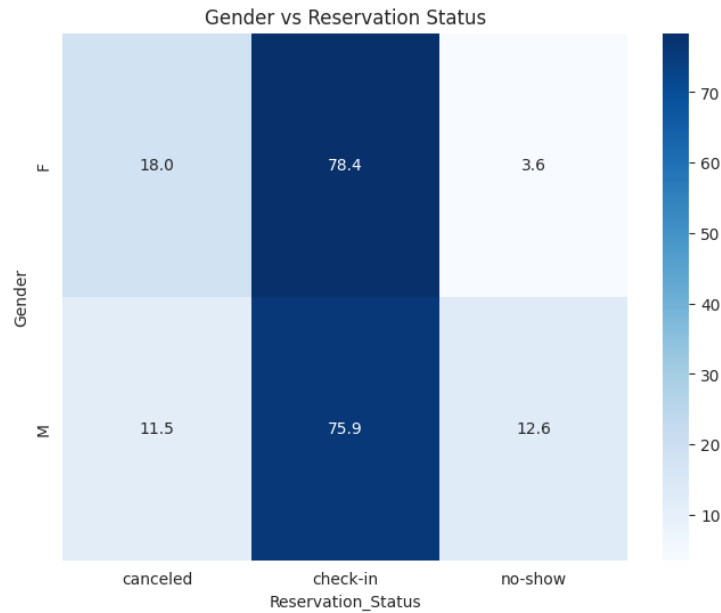


The hotel booking dataset reveals interesting patterns across various customer and booking characteristics. Demographically, there are 12,597 males and 14,902 females, with females slightly outnumbering males in bookings. Ethnically, the dataset includes 7,909 Latinos, 7,766 Caucasians, 7,700 African Americans, and 4,124 Asian Americans, showing a diverse customer base where Asian Americans are the smallest group. Educationally, most customers have a college education (11,924), followed by graduate degrees (9,160), high school (4,613), and middle school (1,802). Geographically, the majority of customers are from the South (10,917), with fewer from the East (5,460), North (5,670), and West (5,452).

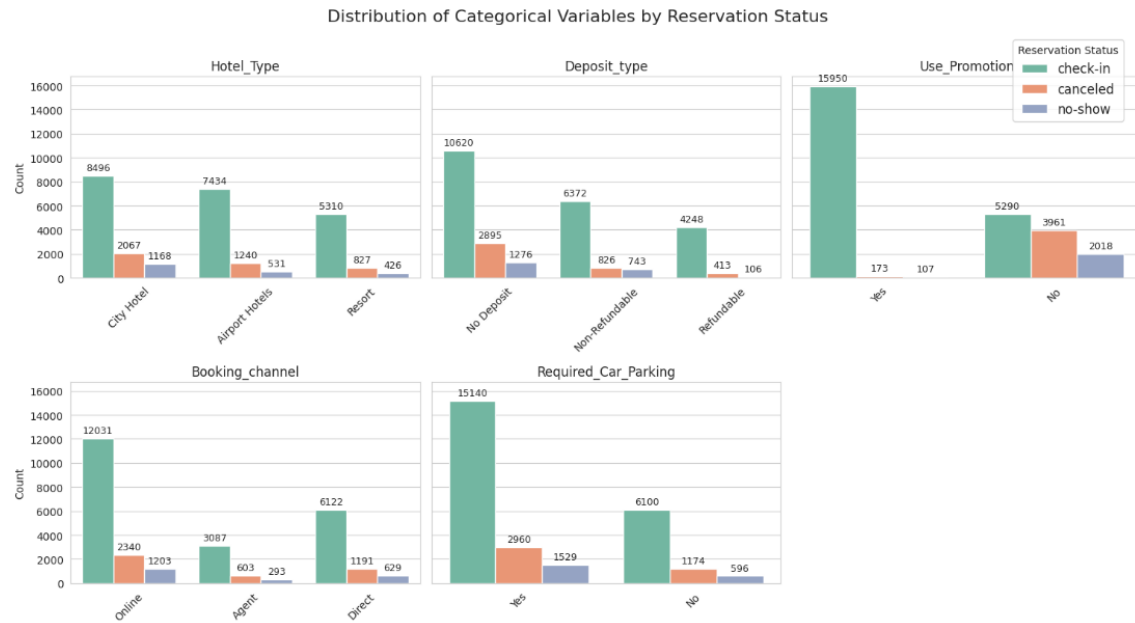
For hotel and booking traits, city hotels lead with 11,731 bookings, followed by airport hotels (9,205) and resorts (6,563). Many customers prefer bed and breakfast (BB) meals (14,224), while 8,081 choose half board and 5,194 opt for full board. A significant 18,144 customers have visited before, compared to 9,155 who haven't, indicating many are repeat visitors, but 24,379 have canceled bookings in the past, with only 3,120 having no history of cancellations, showing cancellations are common.

In booking and payment patterns, most bookings have no deposit (14,791), while 7,941 are non-refundable and 4,767 are refundable. Online bookings dominate with 15,574, followed by direct bookings (7,942) and agent bookings (3,983). Additionally, 19,629 customers require parking, while 7,670 do not need it. For reservation status, 21,210 bookings result in check-ins, but 4,134 are canceled, and 2,125 are no-shows.

Behaviorally and temporally, 16,230 customers used promotions, while 11,269 did not, highlighting the popularity of promotions. Check-ins peak in July (3,002) and August (2,978), with December having the fewest (1,707). Mondays have the most check-ins (4,460), while Saturdays have the fewest (3,024). Age-wise, the 36-45 age group is the largest (5,285), followed by 26-35 (5,163), 46-55 (5,149), 18-25 (4,154), 56-65 (5,114), and 66-70 being the smallest (2,634), reflecting a wide range of customer ages.



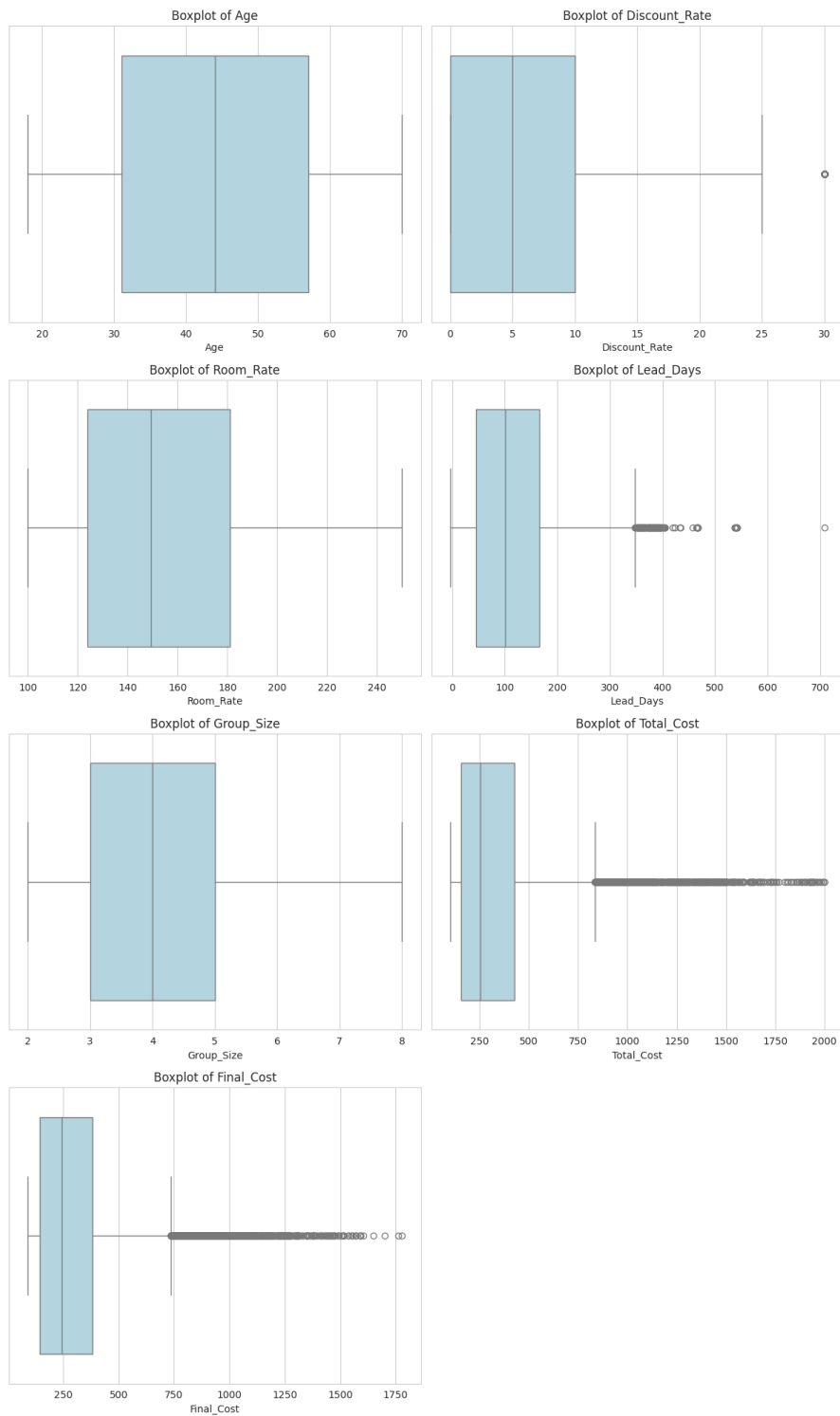
Relationship between gender and reservation status, presenting the percentage distribution of each category. For female customers (F), the majority successfully checked in (78.4%), while 18.0% canceled their reservations, and only 3.6% were no-shows. On the other hand, male customers (M) also had a high check-in rate (75.9%), but they exhibited a lower cancellation rate (11.5%) and a significantly higher no-show rate (12.6%). This shows that while both genders show a strong tendency to check in, male customers are more likely to miss their reservations without canceling, whereas female customers are more proactive in canceling if they do not plan to check in. These insights can help optimize our model by considering gender-based patterns in predicting reservation outcomes, ultimately improving forecasting accuracy and customer engagement strategies.



The charts show how different factors affect reservation status. In Hotel Type, city hotels have the highest check-in numbers, followed by airport hotels and resorts. Deposit Type shows that most people who check in do not pay a deposit, while non-refundable deposits have more cancellations.

Use Promotion indicates that people who use promotions mostly check in, while those who do not use promotions have more cancellations and no-shows. The Booking Channel reveals that most people book online, and direct bookings have fewer check-ins. Required Car Parking shows that customers who need parking check in more often than those who do not. These insights help us understand customer behavior and improve our model.

## Outlier detection and data quality check



Based on the box plots these are the insides that we gathered.

## Age

The boxplot shows a tight interquartile range with zero outliers and the median age appears to be around 45. The age data was distributed between approximately 30 and 50 years. Zero points outside the whiskers, indicating no outliers in the age data.

## Discount Rate

Discount rate is similar to Age, the Discount Rate boxplot has a narrow IQR. No visibility of outliers. The median is exactly on 5, and the data is tightly clustered, suggesting minimal variability and no extreme values. The number 30 is detected as an outlier but in our analysis, we get it as a data point.

## Room Rate

This boxplot also shows a compact IQR with no potential outliers. The median room rate is around 150. With the data ranging roughly between 100 and 240. The distribution appears to be skewed.

## Lead Days

This variable has a noticeable outlier on the upper end, marked by a single point beyond the upper whisker, around 600–700 days. The IQR is narrow, with the median near 100, and IQR ranges between about 0-350, indicating that the outlier represents an unusually long lead time.

## Group Size

The boxplot shows a tight IQR with no outliers for the group size. The median group size is around 4, and the data is concentrated between 2 and 8, suggesting a consistent group size distribution without extreme values.

## Total Cost

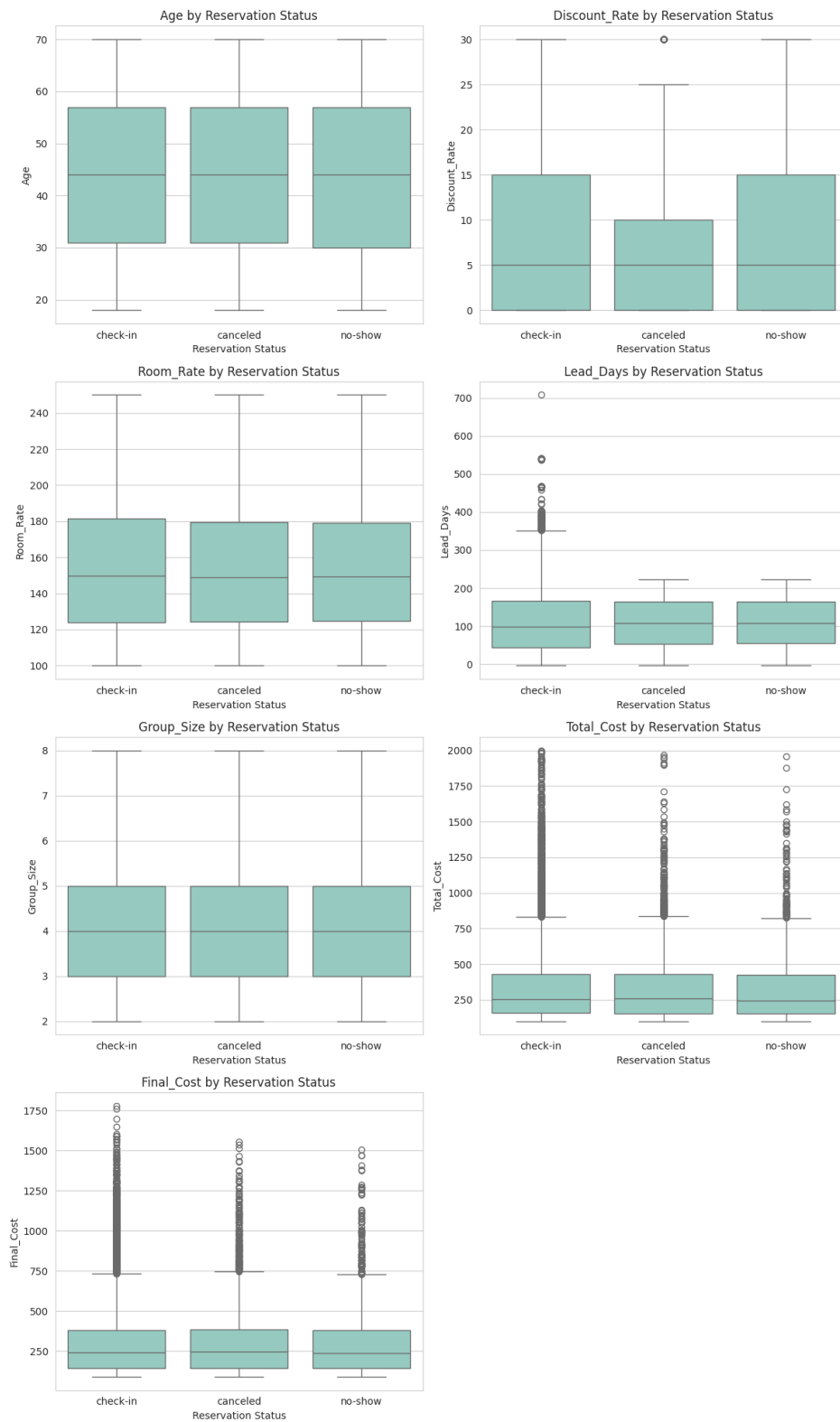
This boxplot has a number of outliers on the upper end, marked by multiple points extending beyond the upper whisker, around 800–2000. The IQR is narrow, with the median near 250, indicating that most costs are lower. But there are a few significantly higher values skewing the distribution. According to our scenario most of the data types that labeled as outliers are valuable for analyzing.

## Final Cost

The boxplot has highly skewed and shows multiple outliers on the upper end. Outliers are extending beyond the upper whisker, around 750–1750. The narrow IQR, and the median near 250. These numbers suggesting that most final costs are lower and there are extremely high-cost cases in bookings.



## Feature Distributions by Reservation Type



### Age by Reservation Status

- The age distribution is relatively similar across check-in, canceled, and no-show. The median age is similar in all reservation status which is about 43 years. There are no significant outliers, age doesn't strongly influence reservation types.

### Discount Rate by Reservation Status

- The discount rate is highest for check-ins while median is around 5% and compared to canceled and no-show reservations they have the same median as the check-ins. This indicates that customers who check in are more likely to receive or utilize with higher discounts and canceled or no-show reservations have lower or no discount rates.

### Room Rate by Reservation Status

- Room rates are consistent and not differences across all three categories, with medians around 150.

### Lead Days by Reservation Status

- Check-ins have a wide range of lead days and detected outliers according to the theory. However, with a median around 100 days and some outliers extending to 700 days. Canceled and no-show reservations have shorter lead days regarding to check-ins.

### Group Size by Reservation Status:

- Group sizes are similar across all categories, with medians around 4 people. The minimal variation, indicate group size doesn't have significant impact on reservation status.

### Total Cost by Reservation Status

- Check-ins have a higher total cost near to 2000. With significant variability and outliers, while canceled and no-show reservations have total costs less than 2000. This higher-cost bookings have a high chance to be a check-ins. lower-cost bookings are more likely to associated with cancellations or no-shows.

### Final Cost by Reservation Status:

- Check-ins have a much higher final cost and distributed around 100-1800. Canceled and no-show reservations have numbers less than about 1600. This reinforces that higher-cost reservations are more likely to be check-ins.

## Quartile Range

- In our analysis, we checked for outliers in the training dataset using the Interquartile Range method to understand the distribution of key features and identify any extreme values. We looked at columns like Age, Discount\_Rate, Room\_Rate, Lead\_Days, Group\_Size, Total\_Cost, and Final\_Cost. For each column, we calculated the first quartile, third quartile, and the IQR, then set the lower bound as 1.5 times the IQR below the first quartile and the upper bound as 1.5 times the IQR above the third quartile. For Age, the bounds were negative 8 and 96, with no outliers found, suggesting the age values were well within a reasonable range.
- Discount\_Rate had bounds of negative 15 and 25, but we found 457 outliers, all at 30 percent, indicating some bookings had unusually high discounts. Room\_Rate showed bounds of 38.50 and 266.50 with no outliers, meaning the room prices were fairly consistent. Lead\_Days had bounds of negative 136.50 and 347.50, with 305 outliers, including values like 367 and 458, reflecting some bookings made far in advance.
- Group\_Size had bounds of 0 and 8, with no outliers, showing typical group sizes. Total\_Cost and Final\_Cost had bounds of negative 249.52 to 834.01 and negative 210.20 to 735.40, respectively, with 1,190 and 1,074 outliers, including high values like 1,458 for Total\_Cost and 1,440 for Final\_Cost, pointing to some expensive bookings. Even though we found these outliers, we noticed they often came from naturally skewed distributions, like higher discounts or longer lead times being somewhat expected behaviors in a hotel booking context. Because of this, we decided not to remove these values, as they still represent realistic scenarios and could provide useful insights for our predictive model.

## Handling Data Quality Issues

- During our analysis, we encountered a significant data quality issue in the training dataset related to the Lead\_Days feature, which represents the number of days between the booking date and the expected check-in date. When we examined rows where Lead\_Days was less than zero, we found several cases where the booking date was actually after the expected check-in date.

For example, one record showed a booking date of July 4, 2015, but an expected check-in date of July 3, 2015, resulting in a Lead\_Days value of negative 1. Another record had a booking date of July 6, 2015, with a check-in date of July 3, 2015, leading to a Lead\_Days value of negative 3. This pattern appeared in multiple rows, with Lead\_Days values like negative 1 or negative 2, indicating that the bookings were supposedly made after the guests had already checked in.

This doesn't make sense in a real-world scenario, as a booking should always happen before the check-in date.

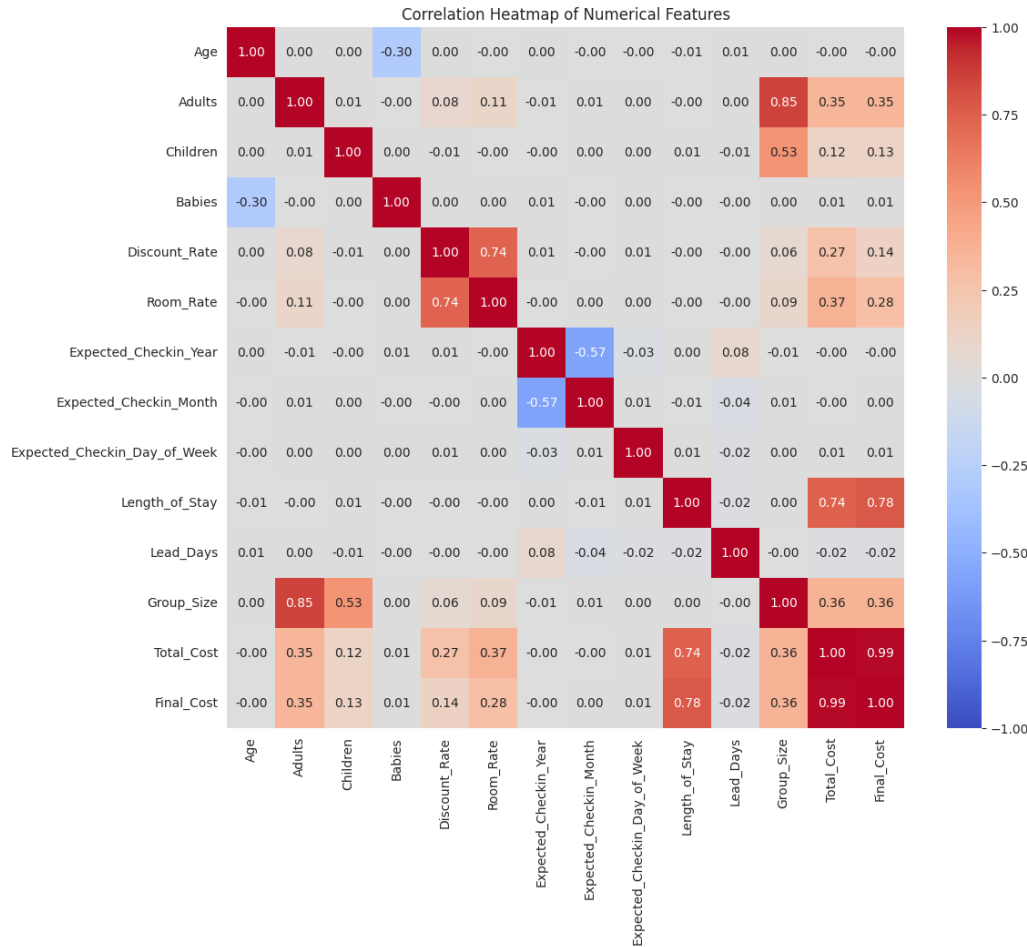
This clearly points to a data quality issue, likely due to errors in how the dates were recorded or calculated. Since our goal in this competition is to predict a customer's reservation status whether they check in, cancel, or don't show up using booking data that's available beforehand, these records with negative Lead\_Days are problematic.

They don't align with the timeline we need for accurate predictions, as a booking made during or after a reservation doesn't fit the predictive context we're working with. This issue highlights the need to either correct these dates or exclude these records to ensure the integrity of our model's training data.

## Correlation Analysis

### Numerical Data

- To identify the relationship between numerical data, we constructed a correlation matrix using Pearson's correlation coefficient.



## Correlation Matrix Insights

### 1. Strong Positive Correlations

- Total Cost & Final Cost (0.99):** These two variables are nearly perfectly correlated, indicating that they represent almost the same information.
- Length of Stay & Final Cost (0.78):** A longer stay tends to increase the final cost.
- Length of Stay & Total Cost (0.74):** Similar to the previous point, longer stays result in higher total costs.
- Room Rate & Discount Rate (0.74):** A higher room rate is associated with higher discount rates, suggesting that discounts are offered proportionally to room prices.
- Adults & Group Size (0.85):** The number of adults is a strong determinant of the total group size.

### 2. Moderate Positive Correlations

- Total Cost & Group Size (0.36):** Larger groups tend to have higher total costs.
- Final Cost & Group Size (0.36):** A similar trend is observed, reinforcing that cost increases with more people.

- c. Total Cost & Discount Rate (0.27): Higher discounts tend to be associated with higher total costs, likely because expensive bookings receive higher discounts.
3. Weak or Negligible Correlations
- a. Most other numerical features show very weak or no meaningful correlation, suggesting that they are either independent or have minimal linear relationships.
4. Negative Correlations
- a. Expected Checkin Year & Expected Checkin Month (-0.57): A negative correlation suggests that months closer to the start of the year might be associated with earlier check-in years.
  - b. Babies & Age (-0.30): This suggests a weak inverse relationship, possibly due to the demographic distribution of bookings.
5. Interpretation & Business Implications
- a. High correlations (like between Final Cost & Total Cost) indicate potential redundancy in variables.
  - b. Length\_of\_Stay is a key factor influencing Total\_Cost and Final\_Cost, making it an essential variable in pricing strategies.
  - c. The Discount\_Rate & Room\_Rate correlation suggests that discounts are more likely on expensive bookings, which could be a strategic pricing policy.

## Feature Selection Analysis

### 1. Numerical Variables Analysis

- We applied the ANOVA (Analysis of Variance) test to identify continuous and discrete variables (with many unique values) that significantly impact Reservation Status.
- The test results are as follows:

| Feature   | F-Score  | P-Value  |
|-----------|----------|----------|
| Lead Days | 3.212942 | 0.040253 |

- Since Lead Days has a P-value  $< 0.05$ , it is statistically significant, meaning it has a measurable impact on reservation status.
- Other continuous variables did not show statistical significance.

## 2. Discrete Variables Analysis

- For discrete variables with a small number of unique values, we performed a Chi-Square test.
- However, none of these variables had a P-value  $< 0.05$ , meaning no significant relationship was found between them and Reservation Status.

## 3. Categorical Variables Analysis

- We applied the Chi-Square test to assess categorical variables and their influence on Reservation Status.
- The test identified several significant features (P-value  $< 0.05$ ), indicating their strong association with reservation status:

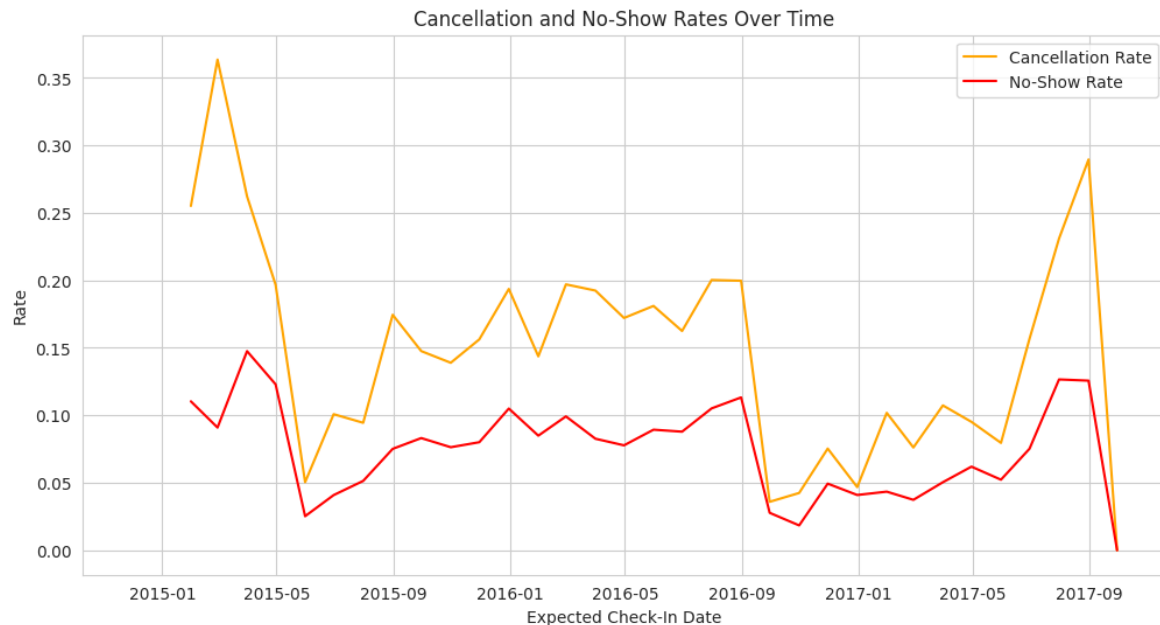
| Feature            | Chi2 Score  | P-Value       |
|--------------------|-------------|---------------|
| Reservation_Status | 6553.374465 | 0.000000e+00  |
| Use_Promotion      | 4053.236423 | 0.000000e+00  |
| Deposit_Type       | 609.999785  | 3.469203e-133 |
| Educational_Level  | 566.080202  | 1.194664e-123 |
| Gender             | 501.588499  | 1.206261e-109 |
| Meal_Type          | 395.025320  | 1.664722e-86  |

## 4. Key Insights & Business Implications

- Lead Days influences Reservation Status, meaning how far in advance a booking is made affects its outcome.
- Use Promotion, Deposit Type, Educational Level, Gender, and Meal Type are all significantly correlated with Reservation Status, suggesting they could be important factors in predicting reservations.
- These insights can be used to refine customer segmentation, marketing strategies, and pricing models.

# Time Series Analysis

## Cancellation and No-Show rates



### 1. Overall Trends and Comparison

The cancellation rate (yellow line) is generally higher than the no-show rate (red line) throughout the period, except for a brief period in late 2015 when the no-show rate surpassed it. Both rates exhibit significant fluctuations, with the cancellation rate showing a wider range of variation (0.10 to 0.35) compared to the no-show rate (nearly 0.00 to 0.15).

### 2. Peaks in Cancellation Rate

The cancellation rate experiences two major peaks, reaching approximately 0.35 in mid-2015 and again in September 2017. These peaks highlight periods of notably high cancellation activity, suggesting potential external influences or shifts in booking behavior.

### 3. Spikes in No-Show Rate

The no-show rate shows significant increases in late 2015 and mid-2017, reaching around 0.15. These spikes indicate times when a higher proportion of expected check-ins failed to occur without prior cancellation, contrasting with the more frequent fluctuations of the cancellation rate.

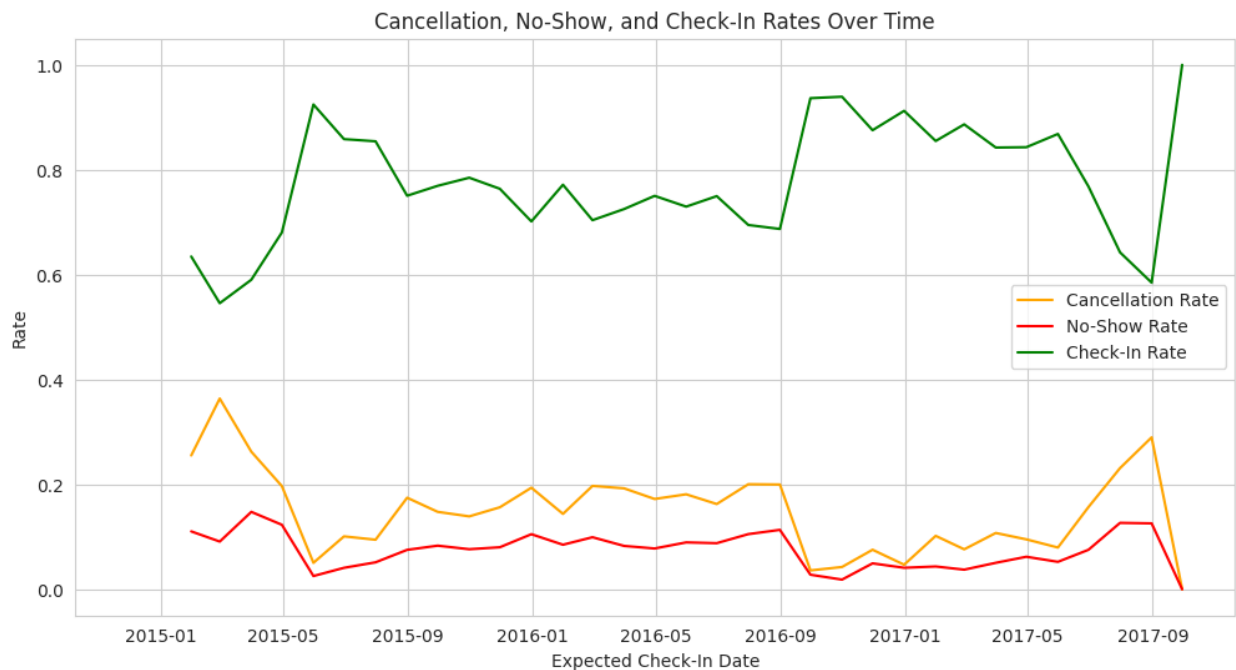


#### 4. Anomalous Dip in No-Show Rate

A striking dip in the no-show rate to nearly 0.00 occurs in mid-2016, standing out as an anomaly compared to the rest of the data. This sharp decline suggests a possible event, policy change, or operational factor that temporarily minimized no-shows.

#### 5. Divergence Between Rates

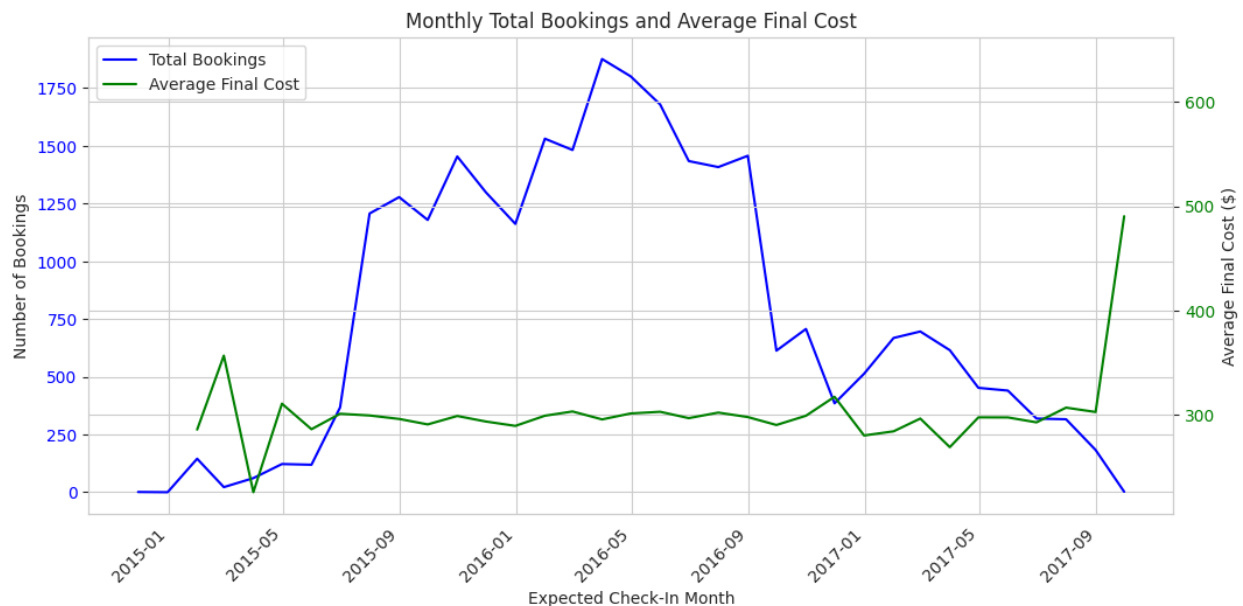
The graph reveals notable periods of divergence between the two rates. For instance, in late 2015, the no-show rate spiked to 0.15 while the cancellation rate declined to 0.10, and in September 2017, the cancellation rate peaked at 0.35 while the no-show rate dropped to 0.05. These divergences suggest that the factors driving cancellations and no-shows may differ or impact them in opposing ways at specific times.



- Graph Overview:

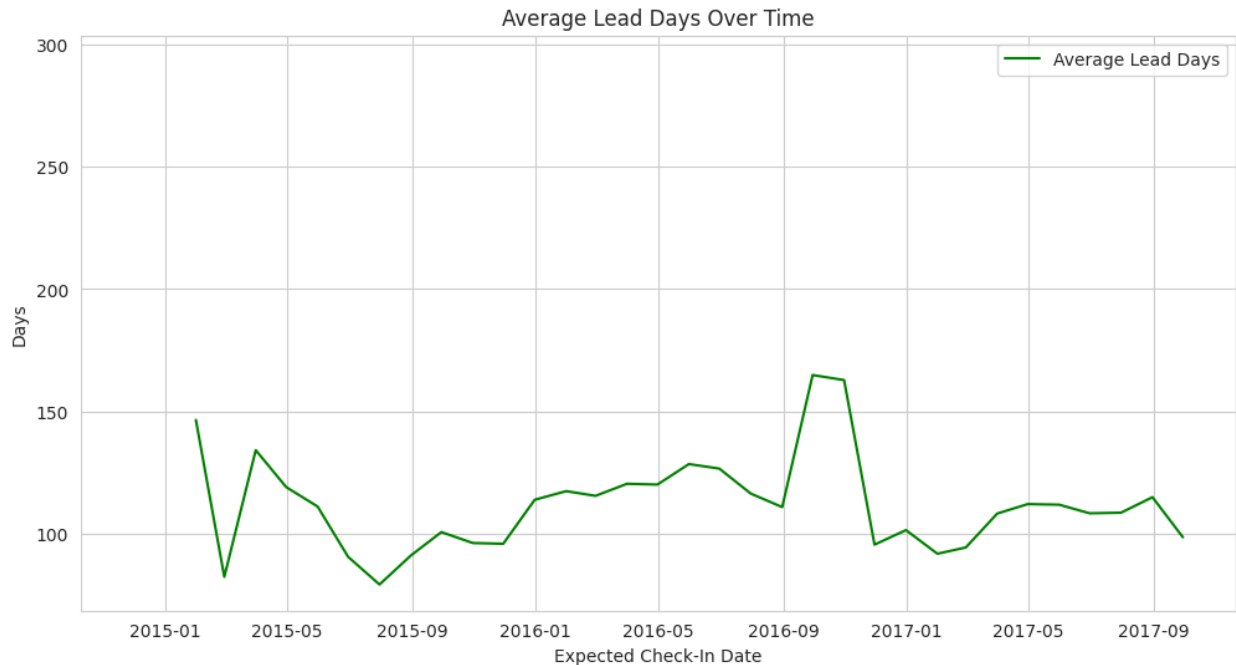
- The image is a line graph titled "Cancellation, No-Show, and Check-In Rates Over Time," showing three rates Cancellation Rate (yellow), No-Show Rate (red), and Check-In Rate (green) from early 2015 to late 2017. The x-axis represents the "Expected Check-In Date," and the y-axis shows rates ranging from 0.0 to 1.0.

- **Check-In Rate Trends**
  - The Check-In Rate (green line) is generally the highest, fluctuating between approximately 0.4 and 1.0. It has notable peaks around mid-2015 and late 2017, followed by a significant drop towards the end of 2017.
- **Cancellation and No-Show Rates**
  - The Cancellation Rate (yellow line) and No-Show Rate (red line) are lower, ranging from 0.0 to 0.2. The Cancellation Rate slightly increases around mid-2015 and late 2017, while the No-Show Rate remains relatively stable and low, occasionally dipping near 0.0.
- **Visual Design**
  - The graph features a light background with grid lines for clarity. The data points are connected by smooth lines, making it easy to track the trends of each rate over time.
- **Comparative Trends**
  - The Check-In Rate consistently exceeds the Cancellation and No-Show Rates, showing more pronounced fluctuations. The graph effectively illustrates how these rates correlate or diverge across the period from early 2015 to late 2017.



- **Rapid Growth in Bookings (Mid-2015 - 2016):** There is a sharp increase in total bookings from mid-2015, peaking in early 2016 before experiencing some fluctuations.
- **Decline in Bookings (Late 2016 - 2017):** After reaching a peak, the total number of bookings shows a significant drop in late 2016 and continues to decline through 2017.

- **Fluctuations in Average Final Cost:** The average final cost experiences some volatility, especially in early 2015 and late 2017, where there are noticeable spikes.
- **Inverse Relationship Between Bookings and Cost:** Generally, when bookings increase, the average final cost remains relatively stable or decreases, and vice versa.
- **End-of-Period Cost Surge:** By late 2017, while bookings continue to decline, the average final cost spikes sharply, suggesting potential changes in pricing strategies, demand shifts, or other influencing factors.



- **Time Frame**
  - The data tracks average lead days from January 2015 to September 2017, covering nearly three years.
- **Measurement Intervals**
  - Data points are recorded every four months (January, May, September), totaling nine observations.
- **Metric Focus**
  - "Average Lead Days" typically measures the time between order placement and fulfillment, indicating supply chain efficiency.
- **Consistency**
  - Regular four-month intervals suggest systematic monitoring of trends over the period.
- **Trend Ambiguity**
  - Without specific numerical values, the direction or magnitude of changes in lead days (e.g., increase, decrease, fluctuations) cannot be determined.

## Analysis by Hotel Type

Since the hotel chain consists of three distinct hotel types—Resort, Airport, and City Hotels—it is expected that there will be variations in reservation behavior based on the type of hotel. Factors such as location, customer demographics, and purpose of travel could contribute to these differences. Therefore, analyzing reservation trends by hotel type helps in understanding customer preferences and optimizing business strategies.

## Reservation Status Distribution by Hotel Type

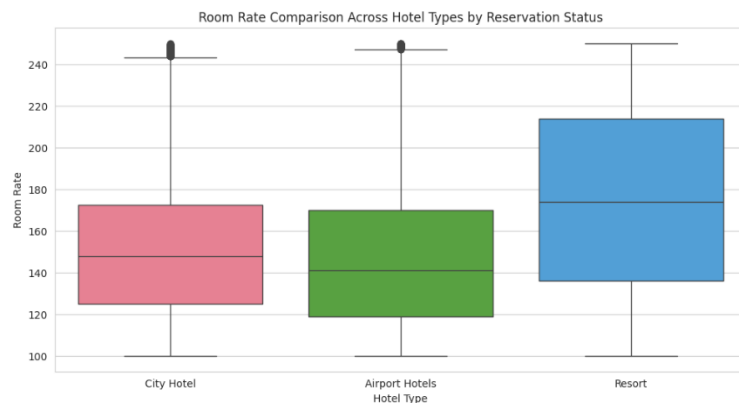
We grouped the data by Hotel Type and Reservation Status to observe trends in cancellations, check-ins, and no-shows.

| Hotel Type     | Canceled | Checked-In | No-Show | Total Reservations |
|----------------|----------|------------|---------|--------------------|
| Airport Hotels | 1,232    | 7,286      | 528     | 9,046              |
| City Hotels    | 2,052    | 8,289      | 1,155   | 11,496             |
| Resort Hotels  | 824      | 5,202      | 425     | 6,451              |

## Key Insights and Business Implications

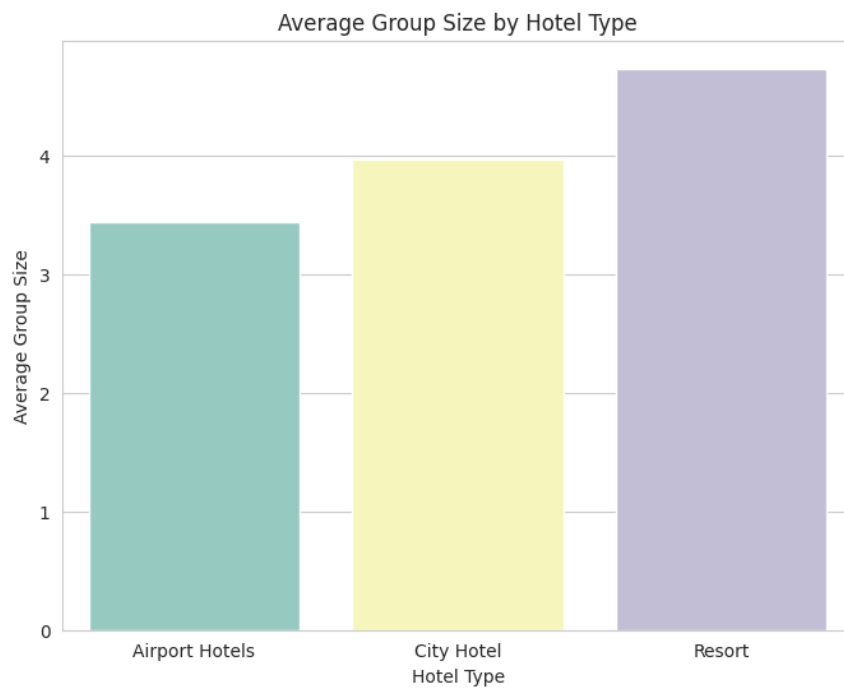
- City Hotels have the highest number of reservations (11,496), followed by Airport Hotels (9,046), and then Resort Hotels (6,451). This suggests that City Hotels are the most popular, likely due to business travelers and tourists preferring central locations.
- Cancellation rates are highest for City Hotels (2,052) compared to Airport Hotels (1,232) and Resort Hotels (824). City Hotels might experience higher cancellations due to flexible booking policies, last-minute travel changes, or business schedule alterations.
- Airport Hotels have the lowest no-show rate (528) compared to City Hotels (1,155) and Resort Hotels (425). This aligns with expectations, as airport travelers tend to have fixed travel plans and are less likely to miss their reservations.
- Resort Hotels have the fewest overall reservations but also the lowest cancellation and no-show rates. Guests booking resorts are likely planning vacations well in advance and are more committed to their stays.

## Room Rate Comparison Across Hotel Types by Reservation Status



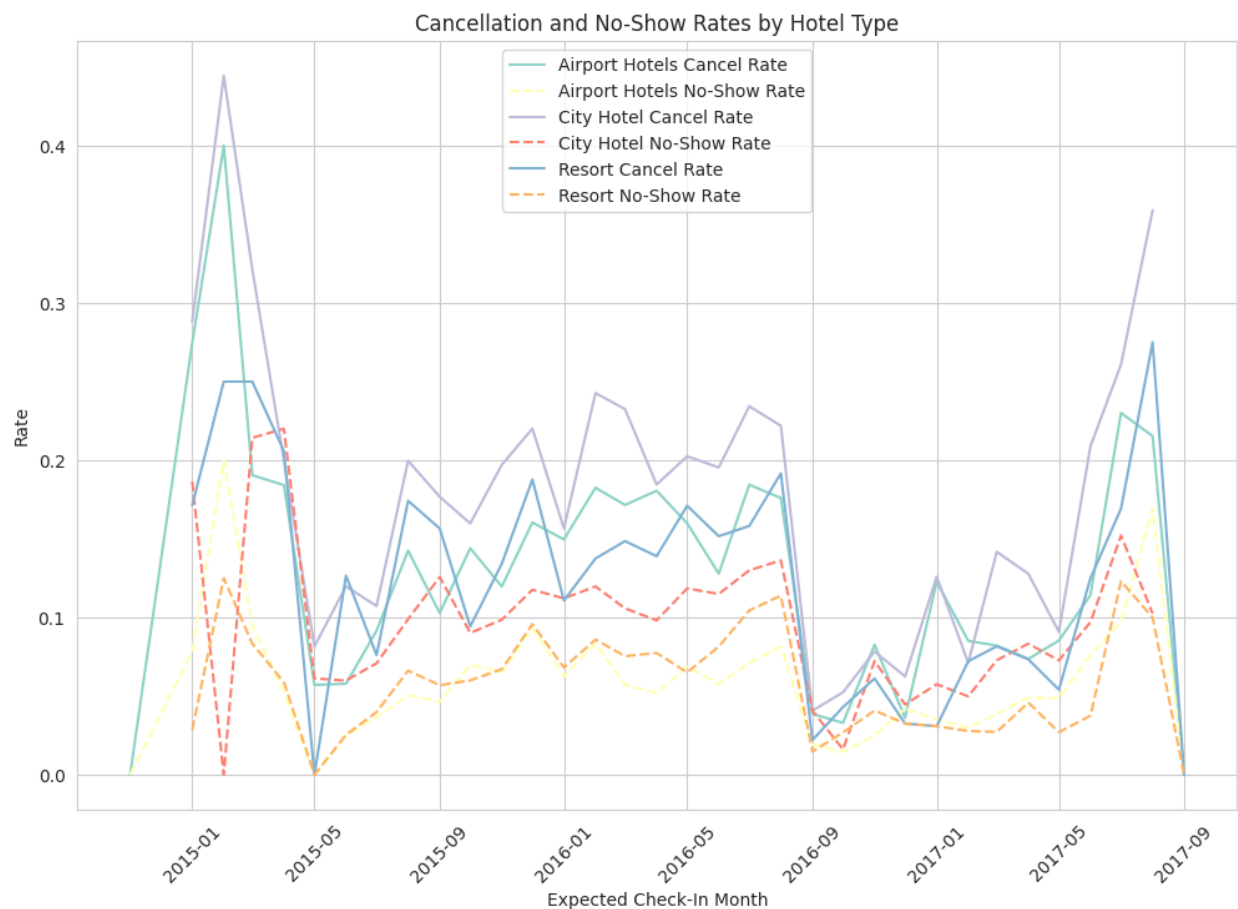
- Based on the box plot, we can conclude that the Room Rate per day for City Hotels and Airport Hotels is approximately equal.
- Additionally, compared to other hotel types, Resort Hotels have a higher Room Rate per day. This can be attributed to the longer Length of Stay in Resort Hotels compared to City and Airport Hotels.

## Average Group Size by Hotel Type



- Based on the bar chart, we can observe the following average group sizes (sum of adults and children) for each hotel type:
- Airport Hotels: Approximately 3 guests per group.
- City Hotels: Approximately 4 guests per group.
- Resort Hotels: Approximately 5 guests per group, representing the largest average group size among the three hotel types.

## Cancellation and No-Show Rates by Hotel Type



Based on the line graph titled "Cancellation and No-Show Rates by Hotel Type," which tracks data from January 2015 to September 2017, here are the key insights presented in a point-wise format:

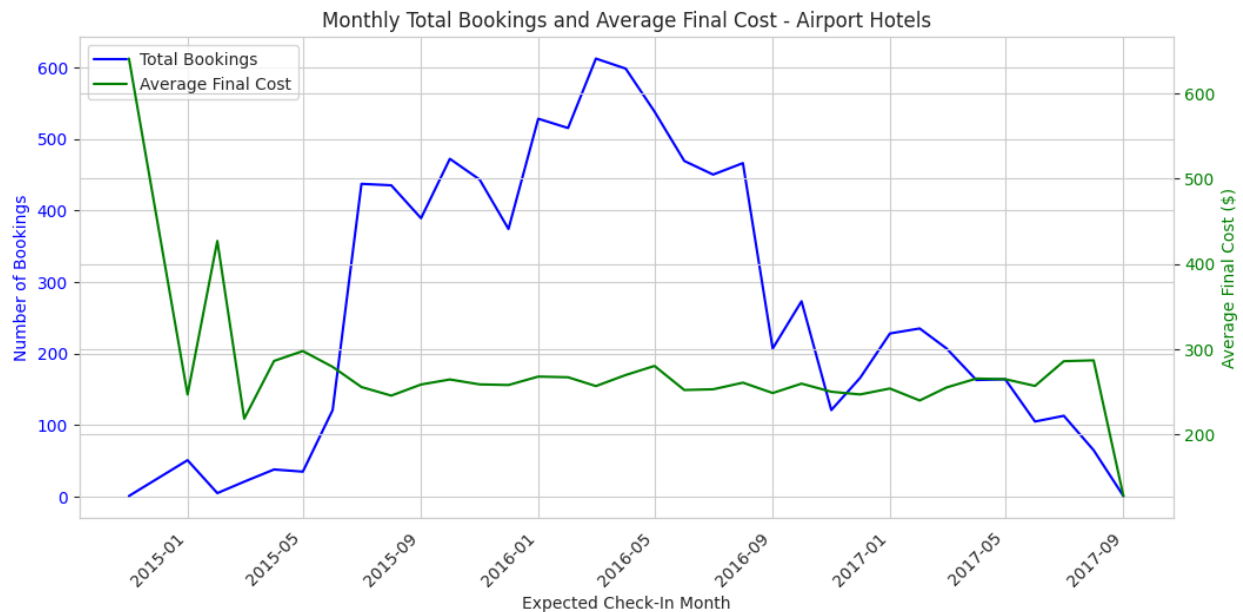
- High Volatility in Early 2015:
  - All hotel types experienced significant peaks in cancellation and no-show rates in early 2015.

- Airport Hotels saw the highest rates, with both cancellation and no-show rates reaching approximately 0.4 (40%).
  - City Hotels had cancellation rates peaking near 0.25 (25%) and no-show rates around 0.25 (25%).
  - Resorts also showed elevated no-show rates, though less extreme, staying below 0.1 (10%) with minor increases.
- Stabilization from Mid-2015 to Mid-2016:
    - After the early 2015 peaks, rates generally declined or stabilized across all hotel types.
    - Airport Hotels no-show rates dropped sharply, remaining below 0.1 (10%) for most of 2016, while cancellation rates fluctuated but stayed lower than the early peak.
    - City Hotels cancellation rates dipped to around 0.1 (10%) in 2016, with no-show rates also staying low (below 0.1 or 10%).
    - Resorts maintained consistently low no-show rates, rarely exceeding 0.1 (10%), showing stability.
- Resurgence in Late 2017:
    - By late 2017, cancellation rates rose again for Airport Hotels (around 0.3 or 30%) and City Hotels (near 0.3 or 30%).
    - No-show rates across all hotel types remained relatively low, with only slight increases (e.g., City Hotels and Resorts showing minor upticks but staying below 0.1 or 10%).
- Hotel Type Comparisons:
    - Airport Hotels displayed the highest volatility, with sharp peaks in early 2015 (0.4 or 40%) and late 2017 (0.3 or 30%) for cancellation rates, and a significant drop in no-show rates after early 2015.
    - City Hotels showed moderate fluctuations, with cancellation rate peaks in mid-2015 (0.25 or 25%) and late 2017 (0.3 or 30%), and low, stable no-show rates post-2015.

- Resorts had the most stable and lowest no-show rates, consistently below 0.1 (10%), with only minor variations throughout the period.
- Possible External Influences:
  - Peaks in early 2015 and late 2017 suggest seasonal factors (e.g., holidays) or external events (e.g., economic shifts, travel disruptions) may have driven higher cancellation and no-show rates.
  - Airport Hotels appear more sensitive to these factors, likely due to reliance on transient travelers and flight schedules.
  - Resorts show resilience with low, stable no-show rates, possibly reflecting longer stays or a different customer base (e.g., vacationers with firmer plans).
- Visual Clarity:
  - The graph uses distinct colors and line styles (e.g., solid teal for Airport Hotels Cancel Rate, dashed orange for Resort No-Show Rate) to clearly differentiate between hotel types and rate categories.
  - A light gray grid background aids in reading precise values, while simple, clear labels (e.g., "Expected Check-In Month" on the x-axis, "Rate" on the y-axis) enhance readability.



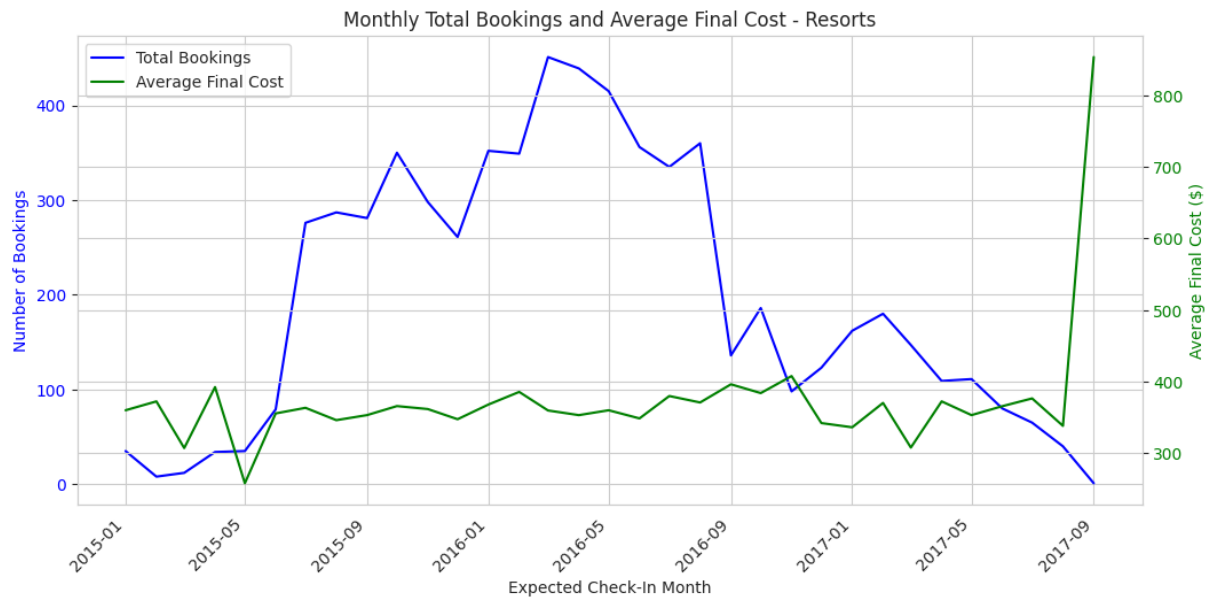
## Monthly Total Bookings and Average Final Cost with Hotel Type



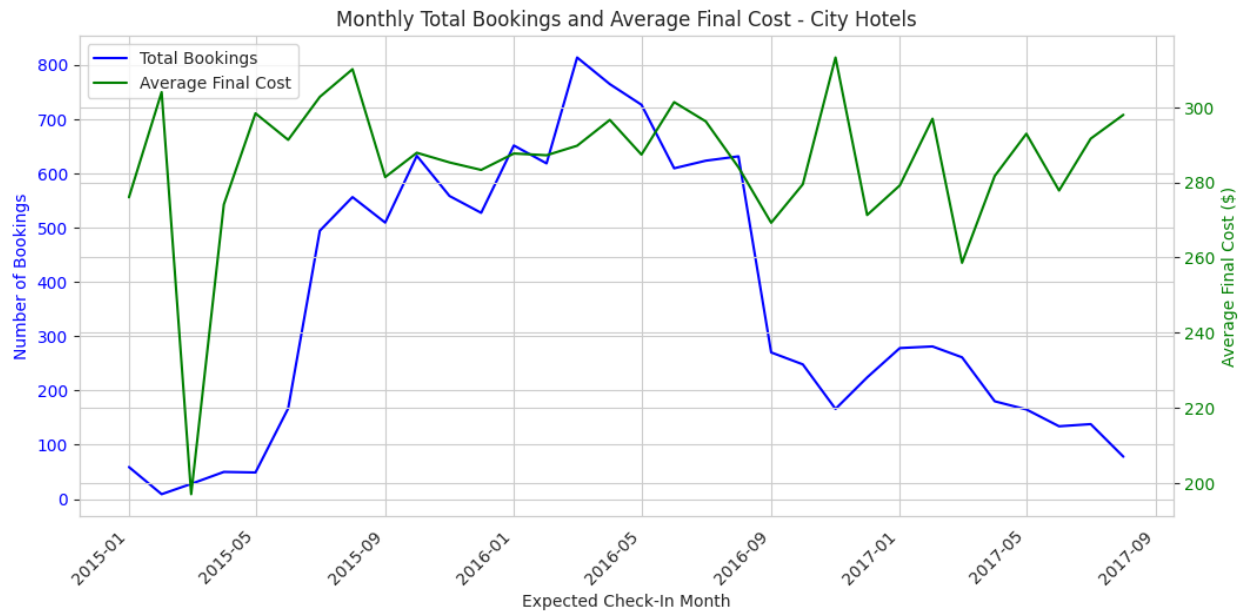
- Initial High Costs with Low Bookings**  
 In early 2015, the average final cost was notably high, around \$500, while total bookings were low, approximately 50. This suggests a period of premium pricing with limited demand, possibly due to seasonal factors, low traveler interest, or market conditions at the start of the timeline.
- Surge in Bookings Following Cost Reduction**  
 From mid-2015 to early 2016, total bookings surged dramatically, peaking at around 550, coinciding with a sharp drop in the average final cost to approximately \$100. This indicates that the significant reduction in prices likely stimulated a substantial increase in demand for airport hotel bookings.
- Stabilization of Average Final Cost Post-2015**  
 After its steep decline in mid-2015, the average final cost stabilized, fluctuating narrowly between \$100 and \$200 through September 2017. Despite significant variations in bookings, the cost remained relatively constant, suggesting a possible shift to a consistent pricing strategy or the establishment of a market equilibrium.
- Volatility in Bookings After the Peak**  
 Following the peak in early 2016, total bookings became highly volatile, dropping sharply to around 100 by mid-2016, rising again to a smaller peak of about 300 in late 2016, and then declining to below 100 by September 2017. This fluctuation may reflect seasonal travel patterns, shifts in consumer behavior, or external influences impacting airport hotel demand.

- Decoupling of Bookings and Average Cost Trends

Early in 2015, there appeared to be a correlation between high bookings and high costs, but this relationship weakened after mid-2015. From that point onward, bookings varied widely while the average cost remained stable, indicating that factors beyond pricing such as competition, travel restrictions, or changes in traveler preferences were likely driving demand.



- Data Period and Metrics: The graph displays data for resorts from January 2015 to September 2017, tracking two metrics: Total Bookings and Average Final Cost.
- Total Bookings Trend: Total Bookings (blue line) fluctuate significantly, reaching a peak of about 400 bookings around mid-2016, then dropping sharply to below 100 bookings by late 2017.
- Average Final Cost Trend: Average Final Cost (green line) stays mostly stable between 300 and 500 dollars until it rises sharply to approximately 800 dollars by September 2017.
- X-Axis Labeling: The x-axis, labeled "Expected Check-in Month," shows monthly data with quarterly markers (e.g., 2015-01, 2015-05), spanning the period from January 2015 to September 2017.
- Graph Legend: A legend identifies the blue line as Total Bookings (left y-axis, 0 to 400 bookings) and the green line as Average Final Cost (right y-axis, 0 to 800 dollars).



- Trend in Total Bookings**  
 The total number of bookings for city hotels starts at a low of approximately 50 bookings in January 2015, rises sharply to a peak of about 800 bookings by May 2015, fluctuates between 600 and 700 bookings through mid-2016, and then steadily declines to around 100 bookings by September 2017.
- Trend in Average Final Cost**  
 The average final cost begins at approximately \$240 in January 2015, climbs to a peak of around \$300 by May 2015, fluctuates between \$280 and \$300 through 2015 and early 2016, and then gradually decreases, stabilizing between \$220 and \$240 by September 2017.
- Correlation Between Bookings and Costs**  
 A clear correlation exists between total bookings and average final cost from 2015 to mid-2016, with peaks in bookings aligning with peaks in costs. However, from mid-2016 onward, while bookings drop sharply, the average final cost declines more gradually, indicating a potential shift in influencing factors later in the period.
- Detailed Monthly Data**  
 The graph provides a granular, month-by-month view of bookings and costs from January 2015 to September 2017, making it a valuable tool for identifying seasonal patterns or monthly trends in the city hotel industry over this timeframe.
- Visual Clarity**  
 The graph is designed for easy interpretation, featuring a blue line for total bookings and a green line for average final cost, a legend in the top right corner, a white background with grid lines, and a centered title, all of which enhance readability and comprehension of the trends.

## Recommendations and Actionable Steps

- City Hotels should implement stricter cancellation policies or offer non-refundable bookings to reduce cancellations. They can also consider offering early check-in incentives to increase actual check-ins.
- Airport Hotels should maintain flexible policies to cater to business travelers who might need last-minute changes. Optimizing pricing strategies can help increase occupancy further.
- Resort Hotels should focus on early bird promotions and package deals to increase bookings. Targeted marketing campaigns for leisure travelers can also help boost occupancy.

This analysis helps in tailoring strategies to maximize revenue, reduce cancellations, and improve customer satisfaction across different hotel types.

## Feature Selection

When we built our predictive model for the hotel chain system to figure out if a customer will check in, cancel, or just not show up, we paid close attention to picking the right features. This step was important because we didn't want to use too many features and risk making the model overfit, where it learns the training data too well but struggles with new data.

We also wanted to keep things manageable and avoid problems that come with having too many variables. So, we dropped `Expected_Checkin_Year` since it didn't show much connection to what we were predicting and wasn't very helpful. We also got rid of `Total_Cost` and kept `Final_Cost` instead, because having both could confuse the model with overlapping information. The `Age` column wasn't needed either, since we'd already grouped ages into ranges with a new feature called `Age_Group`.

We also removed some date columns like `Expected_Checkout`, `Booking_date`, and `Expected_checkin`. Instead of keeping them, we pulled out the most useful details like how many days in advance the booking was made, how long the stay was, and the check-in month and year. This gave us what we needed without cluttering things up.

Features like `Adults` and `Children` were dropped too, since we created a single `group_size` variable to cover the total number of people in the booking. The `Babies` column didn't make the cut either, because it barely related to the outcome and had a lot of missing values.

We also decided to skip `Room_Rate`, since `Final_Cost` which includes discounts and the length of stay tells us more about the guest's financial choice, something that probably affects cancellations or no-shows more directly. Since `Final_Cost` already reflects what `Room_Rate` brings to the table, keeping it made more sense. After all these changes, we ended up with 22 columns, which felt like a solid, focused set to work with for the competition.

## Encoding

After sorting out our features, we moved on to encoding the categorical variables, which is a key step to prepare the data for our models. How we encode these variables and the methods we choose really depend on the type of model we're working with. Since we're testing out multiple models for this competition, we used different encoding techniques to match what each model needs. For some variables that have a natural order or ranking, we went with ordinal encoding.

For example, we mapped `Educational_Level` to numbers like this: Mid-School got 0, High-School got 1, College got 2, and Grad got 3. We did the same for `Income`, where less than 25K became 0, 25K to 50K became 1, 50K to 100K became 2, and more than 100K became 3. `Age_Group` followed a similar pattern, starting with 18-25 as 0 and going up to 66-70 as 5. For simpler yes-or-no features like `Visted_Previously`, `Previous_Cancellations`, `Required_Car_Parking`, and `Use_Promotion`, we kept it straightforward: No became 0 and Yes became 1. `Discount_Rate` also got numbers based on its percentage values, from 0 percent as 0 up to 30 percent as 6.

For the target variable, reservation status, we encoded it into a new column called `Status_Encoded`. Across the training dataset, the counts came out like this: 1 had 20,777 cases, 2 had 4,108, and 3 had 2,108. This gave us a clear numeric way to represent whether someone checked in, canceled, or didn't show up. For other variables that don't have an inherent order, like `Gender`, `Ethnicity`, `Country_region`, `Hotel_Type`, `Meal_Type`, `Booking_channel`, and `Deposit_type`, we used one-hot encoding instead. To avoid creating too many extra columns and keep things efficient, we set `drop_first` to `True`, which drops one category from each variable to prevent overlap. This mix of encoding methods let us tailor the data to suit the different models we're experimenting with, making sure they can pick up patterns effectively for the competition.

When it came to preparing the data for tree-based models, we had to think a bit differently about encoding our categorical variables. Tree-based models, like decision trees or random forests, don't naturally care about hierarchical or ordinal relationships between categories. They're pretty good at handling categorical variables on their own by splitting the data based on the values they see. However, in Python's `scikit-learn` library, these models expect all the input variables to be numeric, even if they can technically work with categories in theory. This meant we needed to convert everything into numbers, but we didn't have to worry too much about preserving the order of things like we did with other models.

So, for both our ordinal columns like `Educational_Level`, `Income`, or `Age_Group` and our nominal columns like `Gender`, `Ethnicity`, or `Hotel_Type`, we applied label encoding. This simply assigns a unique number to each category within a column, turning them into something `scikit-learn` can process. For tree-based models, it doesn't matter if the numbers suggest an order because they don't interpret them that way. They just use the labels to group and split the data based on patterns they find, not assuming that a 2 is "higher" than a 1. This approach kept things simple and practical, letting us use the same numeric format across all variables while relying on the tree models' ability to focus on the distinctions between labels rather than any implied relationships.

## Splitting the dataset

After encoding our variables, we moved on to splitting the dataset, which is a crucial step to train and test our models properly. Our target feature, reservation status, had already been encoded into `Status_Encoded`, with numbers representing whether a customer checked in, canceled, or didn't show up.

Since we're working with multiple types of models for this competition, we handled the splitting process with some flexibility in mind. Specifically, we set things up so the data for tree-based models could be treated a bit differently from the data for other models, like those that need scaling later on.

For this step, we first separated the dataset into features and the target, using `Status_Encoded` as our target variable. Then, we split the data into training and testing sets. Since tree-based models don't require scaling, we made sure their version of the split data was ready to go as is, with the label-encoded variables we prepared earlier. For the other models, which we'll scale later to normalize the numeric features, we kept the split data flexible enough to adjust in the next steps. This approach let us move forward efficiently, setting the stage for training both tree-based and non-tree-based models while keeping our competition goals in sight.

## oversampling

Once we had our dataset split, we turned our attention to dealing with the imbalance in our target feature, `Status_Encoded`, which represents check-ins, cancellations, and no-shows. We noticed a moderate class imbalance, meaning some outcomes like cancellations or no-shows had fewer examples compared to check-ins. To tackle this, we decided to use oversampling, which boosts the number of samples in the minority classes without throwing away any valuable data from the majority class. This felt like the smartest way to give our models a better shot at learning patterns across all outcomes.

Instead of using the standard SMOTE technique, we went with SMOTENC, which is tailored for datasets like ours that mix categorical and numerical features. SMOTENC creates synthetic samples for the minority classes by generating new, realistic data points rather than just copying existing ones. This adds more variety to the data, which we hope will help our models generalize better when they make predictions. By increasing the presence of those underrepresented classes, we're setting things up so the models can pick up on their patterns more effectively, which is especially important for a competition where every bit of performance counts.

## Scaling the data

After oversampling, we moved on to scaling the data, which is another step to get it ready for certain models. For tree-based models, scaling isn't something we need to worry about. Those models treat features as labels and split the data based on thresholds, not distances between values, so the raw numbers work just fine as they are. This means we didn't bother scaling the data for them, keeping that part simple and straightforward.

However, for our linear models and neural network implementations, scaling became important. These types of models rely on calculations that can get thrown off if the features are on different scales, like when one ranges from 0 to 10 and another from 0 to 1000. To fix this, we scaled the numerical columns we had left, specifically `Lead_Days` and `Final_Cost`. By standardizing these features, we made sure they were on a similar level, which helps the linear models and neural networks learn more effectively. This scaled version of the data was set aside just for those models, letting us tailor the preparation to what each approach needed.

# Model training and Evaluation

## Model Training

To train a robust machine learning model to accurately predict the outcome of a customer's booking is the goal of this entire endeavor.

For that we've used multiple architectures and model variations to see how they perform with varying methodologies. As this is a multi-class classification problem, our options for the model were rich in methodologies. We've chosen to test the capacities of tree-based models, linear and neural network-based models and their capacity to accurately predict the class of the testing data.

5 main models of choice were

- Random Forest
- XGBoost
- Linear BGM
- Logistic Regression
- MLP classifier

As we've used different parametric inputs for these models to see their performance, the first choice was to try out using tree-based models. Mainly because of their capacity to handle imbalanced data distributions like ours.

## Random Forest

### 1. Random Forest Without Balancing Class Weights

The first model was trained using a standard Random Forest classifier without any adjustments for class imbalance. The model was trained with 100 trees and a maximum depth of 10.

### 2. Random Forest with Balanced Class Weights

To address class imbalance, the second model was trained with the `class_weight='balanced'` parameter, ensuring that misclassified minority classes were penalized more in the training process.

### 3. Random Forest with feature selection

Feature importance was assessed using the Mean Decrease in Accuracy (MDA) method, which evaluates how much each feature contributes to model performance.

Random Forest model was retrained using only the top 10 features. Our goals here was to

- Reduce model complexity by selecting the top 10 most important features.
- interpretability by eliminating less relevant features.
- Potential reduction in overfitting by removing noise variables.

### 4. Random Forest with Oversampling

To further mitigate class imbalance, we used the previously oversampled dataset that was oversampled with SMOTE as we had a moderate class imbalance. The model was able to learn from a more balanced dataset, reducing bias towards majority classes as SMOTE synthetically generates copies of minority classes.

After running these 4 variants of Random Forest our Results were as follows

| index | Model                                                    | Accuracy | Precision | Recall | F1-Score |
|-------|----------------------------------------------------------|----------|-----------|--------|----------|
| 0     | Random Forest                                            | 0.66     | 0.61      | 0.45   | 0.46     |
| 1     | Random Forest with class balancing                       | 0.70     | 0.65      | 0.68   | 0.64     |
| 2     | Random Forest with class balancing and feature selection | 0.72     | 0.66      | 0.70   | 0.66     |
| 3     | Random Forest with oversampling                          | 0.70     | 0.64      | 0.68   | 0.63     |

## XGBoost

A robust tree based boosted ensemble method, we seeked out the the limits at which we can test tree based model on their ability to work with the complex multi class classification.

For XGB as well, we've used multiple variants of it to see how each model variant performs under different parameters and settings.



## 1. Base XGBoost Model

XGBoost was implemented as an alternative to Random Forest. Since XGBoost requires class labels to be in the format  $\{0,1,2\}$ , the target labels were adjusted accordingly. The model was trained without additional balancing techniques.

## 2. XGBoost with Balanced Class Weights

To handle class imbalance in XGBoost, class weights were computed and applied during training.

## 3 XGBoost with Balanced Class Weights and the top 10 features identified

Feature selection was incorporated using the top 10 features identified from the Random Forest MDA approach.

After running these 3 variants of XGBoost, our results were as following

| Model                                          | Accuracy | Precision | Recall | F1-Score |
|------------------------------------------------|----------|-----------|--------|----------|
| XGBoost                                        | 0.69     | 0.61      | 0.53   | 0.55     |
| XGBoost + balanced weights                     | 0.71     | 0.62      | 0.64   | 0.63     |
| XGBoost + weight balancing + feature selection | 0.71     | 0.63      | 0.66   | 0.64     |

The last tree based model , lightGBM was then trained to see if it can show us some improved performance. It's clear that our F1 score sticks at a certain threshold likely affected by the class imbalance and the similar patterns displayed by no show and cancel classes.

## LightGBM

lightGBM is based on Gradient Boosting Decision Trees (GBDT), where weak learners (decision trees) are iteratively trained to minimize errors.

Instead of evaluating each data point individually, LightGBM bins feature values into discrete groups, significantly improving training efficiency. Unlike XGBoost's depth-wise growth, LightGBM grows trees in a leaf-wise manner, meaning it expands the leaf with the largest loss reduction, leading to faster convergence.

Thus we attempted to use this model to see if there were improvements in performance in the model.

| Model    | Accuracy | Precision | Recall | F1-Score |
|----------|----------|-----------|--------|----------|
| lightBGM | 0.69     | 0.62      | 0.52   | 0.53     |

As lightBGM shows a very similar result to which we got through XGB, the next choice was to try use a linear classifier model. Among multiple models that were all viable to be used for multiclass classification, logistic regression with multinomial loss and lgbfs solver was used.

## Logistic Regression

The model directly predicts the probabilities for all classes and chooses the class with the highest probability. This can work well when there's no strong relationship between the classes, as in your case (hotel types).

It's trained as a single model rather than multiple binary classifiers (like One-vs-Rest). The optimization is done using the softmax function to model the probability of each class.

### 1. Base log reg model

A logistic regression model was trained without handling class imbalance.

### 2. Logistic regression with balanced classes

To address class imbalance, the `class_weight='balanced'` parameter was used, ensuring equal representation of all classes in model learning.

### 3. Logistic regression with balanced classes + feature selection

Using the top 10 features identified through MDA, a logistic regression model was trained with balanced class weights.

- The reduced feature set improved model interpretability.
- Performance was comparable to the previous logistic regression models while reducing model complexity.

### 4. Logistic regression with oversampling

We also used the oversampled dataset which we used by partially oversampling the minority classes as to introduce more datapoints for the model to train on. Thus seeing if that contributed to an increase in model performance.

After running 4 variants of logistic regression models, our results were as following

| Model                                                      | Accuracy | Precision | Recall | F1-Score |
|------------------------------------------------------------|----------|-----------|--------|----------|
| Logistic Regression                                        | 0.69     | 0.62      | 0.51   | 0.53     |
| Logistic Regression + weight balancing                     | 0.71     | 0.66      | 0.70   | 0.65     |
| Logistic Regression + weight balancing + feature selection | 0.71     | 0.66      | 0.70   | 0.66     |
| Logistic Regression + Oversampling                         | 0.70     | 0.63      | 0.59   | 0.59     |

## MLP Classifier

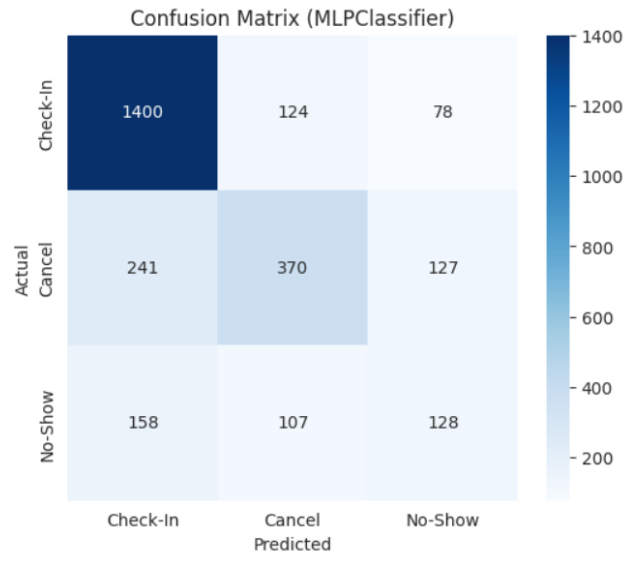
An MLP classifier was implemented as a deep learning approach to classify hotel reservation statuses. The model was trained with the following configuration:

- Hidden layers: (100, 50)
- Activation function: ReLU
- Optimizer: Adam
- Maximum iterations: 500
- Random seed: 42

The deep learning approach allowed for more complex feature interactions, potentially improving predictive performance compared to traditional models.

Model performance

| Model         | Accuracy | Precision | Recall | F1-Score |
|---------------|----------|-----------|--------|----------|
| MLPClassifier | 0.69     | 0.59      | 0.57   | 0.58     |



After running 5 models of different architectural structure and different parameter combinations against them, the f1 score lingered below .70 mark even after applying class imbalance handling and advanced techniques such as SMOTE.

#### Model performances summary

| Model                                                    | Accuracy | Precision | Recall | F1-Score |
|----------------------------------------------------------|----------|-----------|--------|----------|
| Random Forest                                            | 0.66     | 0.61      | 0.45   | 0.46     |
| Random Forest with class balancing                       | 0.70     | 0.65      | 0.68   | 0.64     |
| Random Forest with class balancing and feature selection | 0.72     | 0.66      | 0.70   | 0.66     |
| Random Forest with oversampling                          | 0.70     | 0.64      | 0.68   | 0.63     |
| XGBoost                                                  | 0.69     | 0.61      | 0.53   | 0.55     |
| XGBoost + balanced weights                               | 0.71     | 0.62      | 0.64   | 0.63     |
| XGBoost+weight balanacing+feature selection              | 0.71     | 0.63      | 0.66   | 0.64     |
| lightBGM                                                 | 0.69     | 0.62      | 0.52   | 0.53     |
| Logistic Regression                                      | 0.69     | 0.62      | 0.51   | 0.53     |
| Logistic Regression + weight balancing                   | 0.71     | 0.66      | 0.70   | 0.65     |

|                                                            |      |      |      |      |
|------------------------------------------------------------|------|------|------|------|
| Logistic Regression + weight balancing + feature selection | 0.71 | 0.66 | 0.70 | 0.66 |
| Logistic Regression + Oversampling                         | 0.70 | 0.63 | 0.59 | 0.59 |
| MLPClassifier                                              | 0.69 | 0.59 | 0.57 | 0.58 |

Conclusions after testing our multiple models with different combinations can be listed as,

- The low recall in baseline models (e.g., Random Forest: 0.45, XGBoost: 0.53) strongly suggests the dataset has imbalanced classes, with the minority class being harder to predict.
- Techniques like class balancing, weight balancing, and oversampling improve recall significantly, confirming that addressing imbalance is critical for better performance.
- Adding feature selection slightly boosts performance (e.g., Random Forest: 0.70  $\rightarrow$  0.72, XGBoost: 0.71  $\rightarrow$  0.71), indicating that some features may be noisy or irrelevant.
- However, the small improvement suggests that the feature set is already decently informative, and further gains might require better feature engineering. Yet we've done extensive feature engineering already so it's doubtful
- Random Forest outperforms simpler models (Logistic Regression) and more complex ones (MLPClassifier), suggesting it strikes a good balance between flexibility and robustness for this dataset.
- XGBoost and LightGBM, while powerful, don't excel here, possibly due to insufficient hyperparameter tuning or a dataset that doesn't benefit from gradient boosting's strengths.

The last step of a possible increase in model performance was tuning hyperparameters.

## Hyperparameter Tuning

Since tuning hyperparameters for each and every model run above is computationally expensive, 3 models of different architectures with the highest performances were chosen.

Hyperparams were tuned for Random Forest + weight balanced + feat selection model to see if we can further optimize this mainly because this model was the best among the previous 12 implementations.

Random Search with stratification was used to ensure randomness while we don't sacrifice performance. Stratification was used to create cross validations with equal size in class values.

### 1. Tuning Random Forest + weight balanced + feat selection model

To further optimize performance, hyperparameters such as the number of trees, maximum depth, and minimum samples split were fine-tuned.

- Increased the number of trees for better generalization.
- Optimized depth to balance bias and variance.
- Adjusted minimum samples split to prevent overfitting.

The best model params detected were:

## 2. Tuning MLP Classifier model

Hyperparameter tuning was performed on the MLP model to improve learning efficiency.

The following were the parameters that were tested to find the best features set.

```
param_grid = {
    'hidden_layer_sizes': [(50,), (100,), (50, 50), (100, 50), (100, 100), (200, 100)],
    'activation': ['relu', 'tanh', 'logistic'],
    'solver': ['adam', 'sgd'],
    'alpha': [0.0001, 0.001, 0.01],
    'learning_rate': ['constant', 'adaptive'],
    'learning_rate_init': [0.001, 0.01],
    'max_iter': [500, 1000],
    'early_stopping': [True],
    'validation_fraction': [0.1],
    'n_iter_no_change': [10]
}
```

## 3. Tuning logistic Regression model

The logistic regression model was further optimized using RandomizedSearchCV to find the best hyperparameter combination.

- Adjusted regularization strength (C) using a logarithmic scale.
- Optimized the number of iterations for better convergence.
- Explored class weighting options (None vs. balanced).

Model performances after hyperparameter tuning

| Model               | Accuracy | Precision | Recall | F1-Score |
|---------------------|----------|-----------|--------|----------|
| Random Forest Tuned | 0.71     | 0.63      | 0.65   | 0.63     |
| Tuned MLPClassifier | 0.68     | 0.60      | 0.50   | 0.51     |

Hence, even after doing hyperparameter tuning, we are not seeing any solid improvements of the model.

In our experimentation, we tested over 12 different model combinations and fine-tuned their parameters. After thoroughly evaluating the performance of each model, we selected Random Forest as the champion model due to its superior results in terms of accuracy and robustness. The other major factor in selecting random forest was its interpretability.

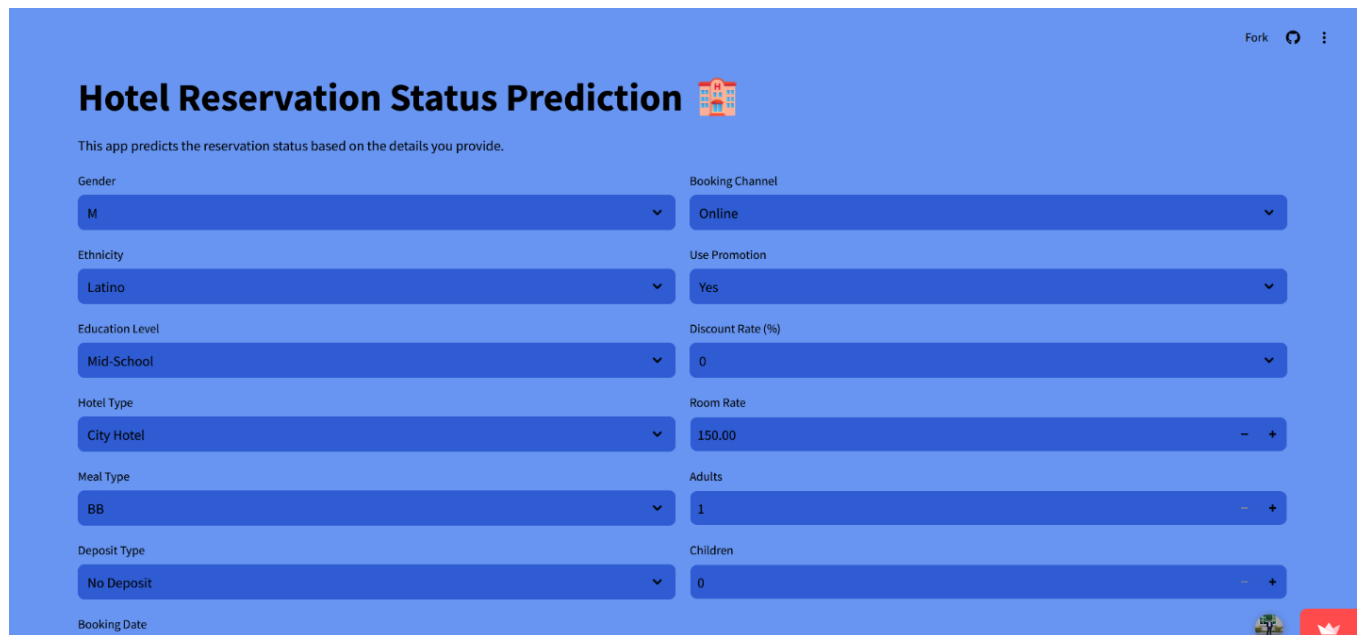
## Model Deployment

As our final champion model of choice was random forest, we serialized the model using joblib and then the model deployment was done with Streamlit to provide us with an interface . Which in turn will allow us to input the data values and get an output on the model's prediction.

The same scaling and encoding done to the model and other preprocessing steps taken while building the model was similarly applied to the input data which we parsed to the model. then this serialized model was loaded and then used to get the predictions as follows.

The streamlit app for predicting the hotel reservations was then deployed to the streamlit community cloud for public access.

Access the public deployment here: <https://hotel-reservation-numlock.streamlit.app/>



The screenshot shows the 'Hotel Reservation Status Prediction' app interface. It features a blue background with a title bar at the top right containing 'Fork', a refresh icon, and a menu icon. The main title 'Hotel Reservation Status Prediction' is followed by a hotel icon. Below the title, a subtitle states: 'This app predicts the reservation status based on the details you provide.' The form consists of two columns of input fields. The left column includes: 'Gender' (dropdown with 'M' selected), 'Ethnicity' (dropdown with 'Latino' selected), 'Education Level' (dropdown with 'Mid-School' selected), 'Hotel Type' (dropdown with 'City Hotel' selected), 'Meal Type' (dropdown with 'BB' selected), 'Deposit Type' (dropdown with 'No Deposit' selected), and 'Booking Date' (text input). The right column includes: 'Booking Channel' (dropdown with 'Online' selected), 'Use Promotion' (dropdown with 'Yes' selected), 'Discount Rate (%)' (dropdown with '0' selected), 'Room Rate' (text input with '150.00' and minus/plus buttons), 'Adults' (text input with '1' and minus/plus buttons), and 'Children' (text input with '0' and minus/plus buttons). At the bottom right, there is a small icon of a person and a red heart icon.

Booking Date

2025/03/03

Expected Check-in Date

2025/03/04

Expected Checkout Date

2025/03/05

Submit

|   | Use_Promotion | Gender | Deposit_type | Ethnicity | Educational_Level | Hotel_Type | Expected_Checkin_Day_of_Week | Booking_channel | Final_Cost | Discount_Rate |
|---|---------------|--------|--------------|-----------|-------------------|------------|------------------------------|-----------------|------------|---------------|
| 0 | 0             | 0      | 0            | 0         | 0                 | 1          | 1                            | 0               | 229.5      | 3             |

The predicted reservation status is: Cancelled

## Customer Segmentation

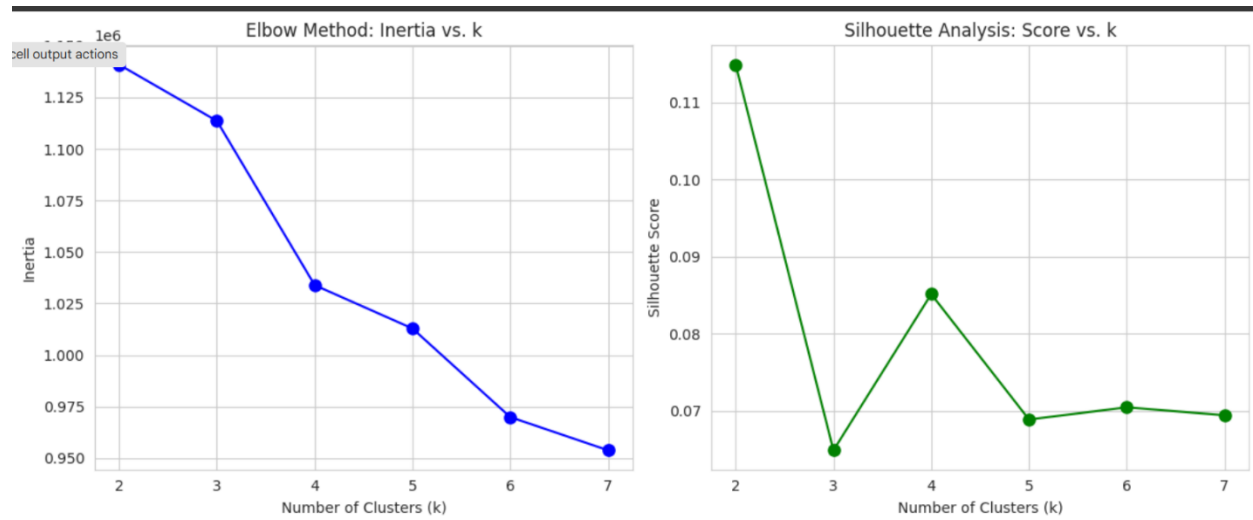
As part of our project, we also developed a customer segmentation model to better understand the different types of guests in the hotel chain dataset. This model helps us group customers into meaningful clusters based on their characteristics, which can guide the hotel in tailoring its services or marketing strategies. To start, we worked with the original dataset, which included columns like Gender, Age, Ethnicity, Educational\_Level, Income, Country\_region, Hotel\_Type, Expected\_checkin, Expected\_checkout, Booking\_date, Adults, Children, Babies, Meal\_Type, Visted\_Previously, Previous\_Cancellations, Deposit\_type, Booking\_channel, Required\_Car\_Parking, Reservation\_Status, Use\_Promotion, Discount\_Rate, and Room\_Rate. Our goal was to preprocess this data, apply clustering, and then analyze the resulting groups to uncover insights.

First, we preprocessed the data to make it suitable for clustering. We created new numerical features from the date columns, calculating Length\_of\_Stay by subtracting Expected\_checkin from Expected\_checkout, and Booking\_Advance\_Days by subtracting Booking\_date from Expected\_checkin. We then used one-hot encoding to transform categorical variables like Gender, Ethnicity, Educational\_Level, Income, Country\_region, Hotel\_Type, Meal\_Type, Deposit\_type, Booking\_channel, Required\_Car\_Parking, Use\_Promotion, Visted\_Previously, and Previous\_Cancellations into numerical format. After that, we dropped the original date columns since we had already extracted the useful information from them. We also left out Reservation\_Status from the clustering features because we wanted the clusters to focus on customer traits, not their booking outcomes. To ensure all features were on the same scale, we applied standard scaling to the data, which is important for clustering algorithms like K-means that rely on distances between data points.

Next, we needed to figure out how many clusters to use. We tested a range of cluster numbers from 2 to 7 and used two methods to decide: the elbow method and silhouette analysis. In the elbow method, we plotted the inertia, which measures how tightly the data points are grouped around their cluster centers, against the number of clusters. The plot showed a sharp drop in inertia from 2 to 4 clusters, but the decrease slowed down after that, suggesting 4 might be a good choice. In the silhouette analysis, we looked at the silhouette score, which measures how similar points are to their own cluster compared to



other clusters. The score peaked at 2 clusters but showed another noticeable peak at 4 clusters before dropping again. Based on both plots, we decided that 4 clusters struck the right balance between having distinct groups and keeping the model manageable, so we went ahead with that for our K-means clustering.

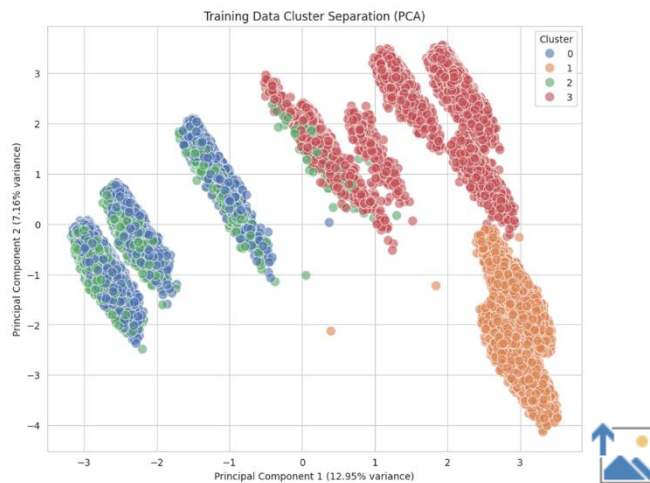


After applying K-means with 4 clusters to the scaled training data, we assigned cluster labels to both the training and test datasets. In the training data, the clusters had sizes of 8,585, 6,562, 4,502, and 7,850 for Clusters 0, 1, 2, and 3, respectively. In the test data, the sizes were 853, 631, 495, and 770 for the same clusters. We then analyzed the clusters by looking at their average values for numerical features like Age, Adults, Children, Babies, Discount\_Rate, Room\_Rate, Length\_of\_Stay, and Booking\_Advance\_Days. For example, in the training data, Cluster 1 stood out with an average of 3 adults per booking compared to around 2 or fewer in the other clusters, and it also had the highest average Room\_Rate at 174.72 and Discount\_Rate at 11.60. This suggests Cluster 1 might represent families or small groups booking more expensive rooms with promotions. Cluster 3, on the other hand, had the lowest number of adults at 1.70 and the lowest Room\_Rate at 148.59, possibly indicating solo travelers or couples opting for budget stays.

We also looked at the percentage distribution of selected categorical features like Gender and Booking\_channel. In the training data, Cluster 1 was almost entirely female at 100 percent, while Cluster 0 was mostly male at 95.1 percent. Cluster 3 was also heavily female at 96.4 percent, and Cluster 2 leaned male at 92.1 percent. This shows a clear gender split across the clusters. For Booking\_channel, all clusters had around 55 to 57 percent of bookings made online, with the rest split between agents and direct bookings, showing no major differences there. When it came to Reservation\_Status, Cluster 1 had the highest check-in rate at 80.9 percent and the lowest no-show rate at 6.5 percent, while Cluster 0 had the highest no-show rate at 10.1 percent. In the test data, the patterns were similar but with lower check-in rates overall, like Cluster 1 at 63.9 percent, and higher no-show rates, like Cluster 2 at 19.4 percent, possibly reflecting some differences in the test set's behavior.

To visualize the clusters, we used PCA to reduce the data to two dimensions, capturing 12.95 percent of the variance with the first component and 7.16 percent with the second. While this only explains part of the data's variation, the scatterplot still helped us see how the clusters separated, confirming they were distinct enough to be meaningful. Overall, this segmentation revealed useful patterns, like identifying

budget-conscious solo travelers in Cluster 3 and promotion-seeking family groups in Cluster 1, which the hotel chain can use to target specific customer needs.



## Conclusion

This study aimed to enhance Hotel A's revenue management by analyzing booking cancellations and no-shows through exploratory data analysis and predictive modeling. We tested multiple machine learning models, including XGBoost, logistic regression, and others, to classify reservation outcomes (Check-In, Cancel, No-Show), optimizing for the F1-score as the primary performance metric. Despite extensive feature engineering, preprocessing adjustments, and model comparisons, the highest achievable F1-score across methods was approximately 0.65.

This performance plateau reflects a significant challenge in distinguishing between the No-Show and Cancel classes, which exhibit smaller record counts and share similar properties—such as lead time, booking channel preferences, and guest demographics—making separation difficult.

These findings suggest that while predictive analytics offers valuable insights for operational planning, the inherent overlap and data limitations constrain classification precision for minority classes. Hotel A's management may benefit from supplementing these models with targeted policy interventions to address no-shows and cancellations effectively.